

Boring Financial 智能账单分类系统

项目成员： 郑博文、林子尧、单彬、沈柔希

项目周期： 2026 年 3 月 – 2026 年 6 月

目录

第一章 项目概述	1
1.1 项目背景与问题分析	1
1.2 项目目标	1
1.3 项目范围	3
1.3.1 范围内功能	3
1.3.2 范围外内容	4
1.4 项目收益	4
第二章 可行性分析与规划	5
2.1 开发流程与选择依据	5
2.1.1 迭代阶段与反馈闭环	5
2.2 可行性分析	7
2.2.1 方案选型分析	8
2.3 项目计划与进度	8
2.3.1 工作分解结构 (WBS)	8
2.3.2 里程碑计划	9
2.3.3 进度甘特图	10
2.4 团队组织与协作	10
2.4.1 团队成员与分工	10
2.4.2 人员变动预案	11
2.4.3 协作机制	11
2.5 风险管理	12
2.5.1 变更管理策略	13
第三章 需求分析	14
3.1 需求获取方法	14
3.2 利益相关者分析	15
3.3 功能需求	15
3.4 非功能需求	17
3.5 用户场景分析	17
3.5.1 普通用户场景	17
3.5.2 家庭组织场景	18

3.5.3	系统运维场景	18
3.6	异常场景处理	19
3.7	用例建模	19
3.8	业务流程建模	21
第四章	用户界面与体验设计	22
4.1	设计系统	22
4.2	主要页面原型	23
4.2.1	登录与注册页面	23
4.2.2	仪表盘页面	23
4.2.3	账单导入页面	24
4.2.4	交易列表页面	24
4.2.5	分类校正工作台	25
4.2.6	分类管理页面	25
4.2.7	消费人格页面	26
4.2.8	家庭组织页面	26
4.2.9	报表中心页面	27
4.2.10	系统设置页面	27
4.3	前端架构与组件职责	28
4.4	用户体验目标	28
第五章	系统设计	30
5.1	总体架构	30
5.2	技术选型	31
5.3	前端架构	31
5.4	后端架构	33
5.5	数据隔离策略	33
5.6	部署架构	33
5.7	安全机制	35
第六章	程序开发	36
6.1	面向对象程序组织	36
6.1.1	领域实体及其关联关系	36
6.1.2	分类器策略	38
6.1.3	账单解析器	38
6.1.4	报表构建器	39
6.2	数据库设计	40

6.2.1	核心账单与分类	40
6.2.2	家庭组织与报表扩展	41
6.3	设计模式应用	42
6.4	核心流程设计	43
6.4.1	账单导入与大模型初分	43
6.4.2	家庭组织聚合查询	44
第七章	验收与发布	45
7.1	自动化测试	45
7.2	测试结果	46
7.3	验收测试	46
7.4	发布状态	48
第八章	运行与维护	49
8.1	运行环境	49
8.2	性能管理	50
8.3	可靠性保障	50
8.4	运维检查	51
第九章	总结与改进计划	53
9.1	项目完成情况	53
9.2	当前局限与后续改进	53

第一章 项目概述

1.1 项目背景与问题分析

在个人财务管理场景中，自动记账工具（如微信账单导出、支付宝账单导出）已成为日常财务整理的重要手段。然而，传统记账软件在自动分类环节存在显著短板，直接影响统计分析的准确性。一个典型且普遍的场景是：当用户向校园一卡通充值时，账单上的商户名称可能仅显示为「东南大学」。基于关键词匹配的传统分类引擎很容易将其归入「学习」类别，但从真实消费意图来看，这笔交易更接近于「餐饮/校园卡充值」。这种语义误判一旦发生，不仅会扭曲用户的消费结构认知，还会在没有校正机制的情况下持续污染后续的财务统计。

进一步分析，个人用户在整理微信、支付宝账单时通常会遇到三类共性问题：

1. **自动分类依赖浅层规则**：传统分类引擎仅根据商户名或关键词做简单匹配，难以理解「在便利店买口罩」和「在便利店买零食」之间的消费意图差异。
2. **分类依据不透明**：用户不知道一笔交易为什么被归入某个类别，缺乏解释性，因而无法判断分类是否可信。
3. **误判缺少校正入口**：当分类出错时，用户要么被动接受错误结果，要么需要逐个手动修改，缺乏一个高效、可控的人工校正机制。

Boring Financial 正是针对上述问题设计的解决方案。系统围绕「导入—大模型语义初分—人工校正—多维度分析—报表输出」的完整闭环展开，将原始账单转换为语义准确、可解释、可修正的财务分析结果，同时引入消费人格画像和家庭组织聚合等创新分析维度，为用户提供比传统记账工具更深层的财务洞察。

1.2 项目目标

Boring Financial 的核心定位是一个面向个人与家庭的智能账单分类与财务分析系统。项目的具体目标如表 1.1 所示。

表 1.1 项目目标

目标	说明
完成记账分析闭环	支持账单导入、格式解析、大模型语义初分、人工校正、统计看板、消费人格画像、家庭组织聚合和 PDF 报表的完整流程。
支持多用户与数据隔离	用户默认只能访问自己的账单、交易、分类和报表；加入家庭组织后可按成员角色查看聚合数据，个人数据的独立性不受影响。
支持语义分类与人工校正	以大模型语义分类为主要手段，同时保留规则分类作为兜底方案，并为低置信度交易提供人工校正入口，确保用户对最终分类结果拥有控制权。
支持工程化部署	提供清晰的系统架构、接口文档、开发指南、测试套件和部署说明，支持从本地开发到 Docker Compose 裸机部署的多种运行模式。
控制实现复杂度	聚焦记账分析闭环本身，不引入支付、清算和银行级风控等超出项目范围的功能。

1.3 项目范围

1.3.1 范围内功能

表 1.2 项目范围内功能模块

模块	范围说明
用户认证	注册、登录、JWT 鉴权、多用户数据隔离。
账单导入	支持微信、支付宝账单文件上传，管理导入批次状态和导入记录。
账单解析	将不同平台的原始账单格式转换为统一的交易数据结构，支持去重。
大模型初分	支持规则分类、缓存分类和外部大模型语义分类的组合链路，提供分类理由与置信度，含重试队列机制。
人工校正	用户可筛选和复核低置信度交易，修正错误分类，修正结果在后续统计中体现。
分类管理	支持系统预设分类和用户自定义分类，系统分类不可禁用。
财务看板	展示收入、支出、结余、储蓄率、分类占比分布、Top 商户和收支趋势。
消费人格	基于行为经济学理论生成 16 型消费人格画像、多维财务健康评分，并提供自评问卷与数据画像的偏差对比。
家庭组织	支持创建家庭组织、管理成员角色 (owner/admin/member)，聚合家庭 Dashboard、家庭消费人格和家庭报表。
PDF 报表	按日期范围和导入文件生成可下载的 PDF 财务报告。
系统部署	支持本地开发运行、Docker Compose 编排部署和裸机 Nginx + systemd 部署。

1.3.2 范围外内容

表 1.3 项目范围外内容及原因

不做内容	原因
支付、转账、清算	涉及真实资金流和金融合规要求，超出项目范围。
银行级风控与审计	需要严格的合规流程和高安全保障，不属于记账分析闭环的核心关注点。
多机构财务管理	当前定位为个人与家庭的账单分类与分析工具。
复杂家庭预算审批流程	当前家庭组织模块聚焦成员聚合和角色管理，不做多级审批流。
商业化运营后台	系统聚焦个人使用和轻量部署，不覆盖商业运营能力。

1.4 项目收益

Boring Financial 的收益可以从多个利益相关方视角进行审视，如表 1.4 所示。

表 1.4 项目收益分析

受益方	收益
普通用户	减少误分类带来的统计偏差，快速理解个人支出结构、消费趋势和消费行为特征。通过消费人格画像获得对自身消费心理的深层认知，通过家庭组织模块了解家庭成员的整体财务状况。
家庭管理员	可将个人账本聚合为家庭视图，了解家庭整体的收入支出情况和消费结构，无需为家庭成员创建共享账本或手动合并数据。
系统运维人员	基于清晰的模块边界、标准化的接口文档、自动化测试套件和多种部署方案，可快速搭建和维护系统运行环境。
项目开发团队	在实践中覆盖了需求分析、系统设计、编码实现、测试验证、部署运维和文档整理的全流程，积累了从单体应用到前后端分离系统的完整开发经验。

第二章 可行性分析与规划

2.1 开发流程与选择依据

本项目采用轻量级迭代开发流程，而非传统的瀑布模型。这一选择基于以下实际约束和工程考量：

表 2.1 开发流程选择依据

因素	说明
有限的项目周期	项目从 3 月持续至 6 月，需要在约四个月内完成从需求分析到部署交付的全部工作，要求流程具备快速反馈和持续交付能力。
需求随实现迭代演化	不同平台的账单格式差异、大模型分类的实际效果以及前端交互体验都需要在实际实现和联调过程中不断验证和修正，难以在初始阶段完全冻结需求。
团队可并行推进	前端、后端、分类模型、测试和文档等工作可按照模块边界并行开发，迭代流程天然支持这种并行协作模式。
MVP 优先策略	首先保证「导入—大模型初分—人工校正—分析—报表」的核心闭环可运行，后续再逐步补充增强功能。
文档持续积累	各阶段的过程文档在开发过程中持续积累，在最终交付阶段统一整理为完整报告，避免期末集中赶工。

2.1.1 迭代阶段与反馈闭环

系统的开发过程划分为四个阶段，阶段之间通过反馈回路进行修正：

- **阶段一（3 月）：选题与需求。**确立项目目标、完成可行性研究、明确团队分工和需求范围。核心产出包括项目目标说明、可行性论证、需求列表和利益相关者分析。
- **阶段二（4 月）：原型与架构。**完成前端页面原型、系统架构设计、组件图、部署图，以及安全性和性能的初步设计。此阶段的架构决策为后续实现提供了稳定的技术骨架。
- **阶段三（5 月）：功能实现与联调。**完成核心功能的开发并进行前后端联调。实现范围覆盖用户认证、账单导入、AI 分类、人工校正、Dashboard、消费人格画像、

家庭组织和 PDF 报表。实现过程中发现的技术约束会反向修正架构设计，联调中发现的问题会反向修正需求定义。

- **阶段四（6 月）：测试、部署与报告。**完成自动化测试和手工验收测试，编写部署文档，整理最终报告和答辩材料。测试中发现的缺陷会触发对实现和设计的修正。

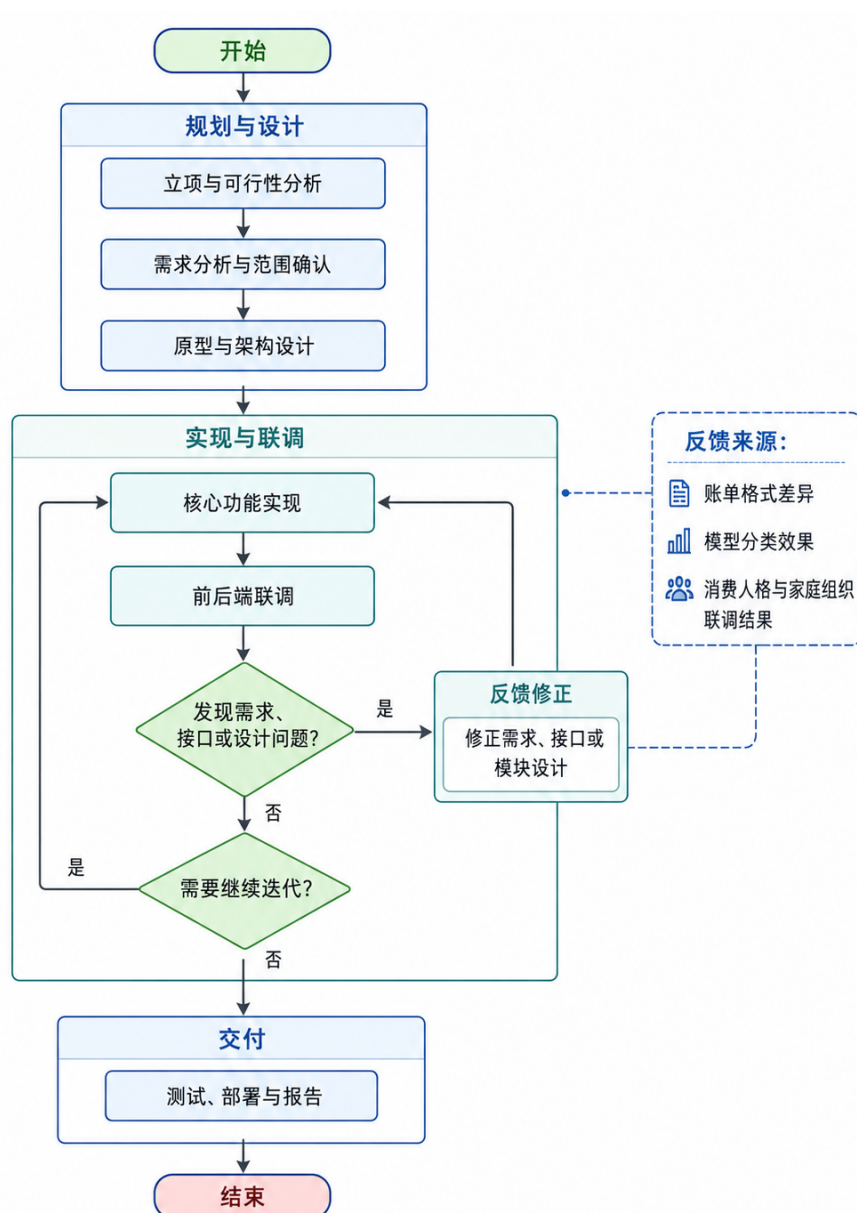


图 2.1 开发流程迭代图

2.2 可行性分析

在项目启动阶段，从五个维度对项目可行性进行了系统评估。评估结果如表 2.2 所示。

表 2.2 多维度可行性分析

维度	结论	依据
业务可行性	可行	自动记账分类误判是真实的用户痛点，大模型语义分类结合人工校正的解决方案价值明确。消费人格画像和家庭组织聚合进一步增强了产品的差异化竞争力。
技术可行性	可行	技术栈选择成熟稳定：FastAPI 提供高性能异步 API，Vue 3 + Element Plus 提供组件化的前端能力，SQLAlchemy 提供 ORM 层的数据操作抽象，Docker 提供一致化的运行环境。所有技术均有丰富的社区支持和生产实践验证。
进度可行性	可行	四个月的项目周期可以支撑四阶段的迭代开发节奏。四人团队按前端、后端、模型/测试的模块边界并行推进，核心闭环可以在两个月内完成 MVP。
资源可行性	可行	开发设备使用团队成员的本地电脑，无需额外硬件投入。Git、Docker、PostgreSQL、Redis 等工具均采用开源或免费方案。模型服务支持本地 mock 和规则兜底，可灵活控制对商用 API 的依赖。
经济可行性	可行	项目预算极低，主要成本为成员的开发时间和可选的大模型 API 调用费用（可通过规则兜底和本地模型服务进一步压缩）。
维护可行性	基本可行	采用 monorepo 代码组织、分层后端架构和前端组件化设计，配合接口文档和自动化测试，为后续维护提供了可追溯的技术基础。

2.2.1 方案选型分析

在系统核心分类策略上，项目初期评估了三种备选方案，如表 2.3 所示。

表 2.3 分类策略方案对比

方案	优点	缺点	结论
仅关键词规则分类	稳定可控，实现简单	难以理解「东南大学饭卡充值」等语义场景，误判后统计失真	不作主要方案
仅大模型自动分类	语义理解能力强	用户缺少最终控制权，模型错误直接污染统计	不作完整方案
大模型初分 + 人工校正	兼顾语义理解、可解释性和用户控制权	实现复杂度更高	最终方案

最终选择「大模型语义初分 + 人工校正」的前后端分离架构，同时保留规则分类作为兜底方案和重试队列作为容错机制。该方案在自动分类的语义准确性与用户对关键错误的最终控制权之间取得了平衡。

2.3 项目计划与进度

2.3.1 工作分解结构（WBS）

项目的核心任务分解如表 2.4 所示。各任务按模块边界划分，明确负责人、前置依赖和交付物。

表 2.4 项目工作分解结构（WBS）

编号	任务	负责人	前置依赖	交付物
WBS-01	项目目标与范围确认	郑博文	无	项目立项与可行性资料
WBS-02	需求分析与规格说明	郑博文、全体	WBS-01	需求规格文档
WBS-03	后端数据模型与接口设计	林子尧	WBS-02	ORM 模型、API 路由定义

接下页

表 2.4 项目工作分解结构（续）

编号	任务	负责人	前置依赖	交付物
WBS-04	账单解析与分类服务	林子尧、沈柔希	WBS-03	Parser、Classifier 实现
WBS-05	前端页面与交互实现	单彬	WBS-02、WBS-03	全部业务前端页面
WBS-05A	家庭组织协作能力	林子尧、单彬	WBS-03、WBS-05	Organization API 与页面
WBS-06	模型 provider 与重试机制	沈柔希	WBS-04	Provider 配置、重试队列
WBS-07	前后端联调	全体	WBS-03、WBS-05	可运行系统 demo
WBS-08	测试与质量保证	沈柔希、全体	WBS-07	pytest 报告、验收记录
WBS-09	部署与运行文档	沈柔希、林子尧	WBS-07	部署说明文档
WBS-10	最终报告与答辩材料	郑博文、全体	WBS-01–WBS-09	最终报告、答辩 PPT

2.3.2 里程碑计划

项目设置了五个关键里程碑节点，对齐各阶段的验收要求，如表 2.5 所示。

表 2.5 项目里程碑计划

里程碑	时间	阶段	成功标准
M1	4 月上旬	阶段一	项目名称、团队、可行性、进度和需求分析可展示
M2	4 月下旬	阶段二	用户体验设计、系统架构、安全和性能设计可说明
M3	5 月中旬	阶段三	IDE 配置、组件复用、类图、顺序图和设计模式可解释
M4	5 月下旬	阶段三/四	测试覆盖率、单元/集成/验收测试、可靠性设计可验证
M5	6 月	阶段四	系统 demo、测试结果、部署资料和最终报告完整交付

2.3.3 进度甘特图

图 2.2 展示了项目从 3 月到 6 月的整体进度安排和各阶段的并行关系。

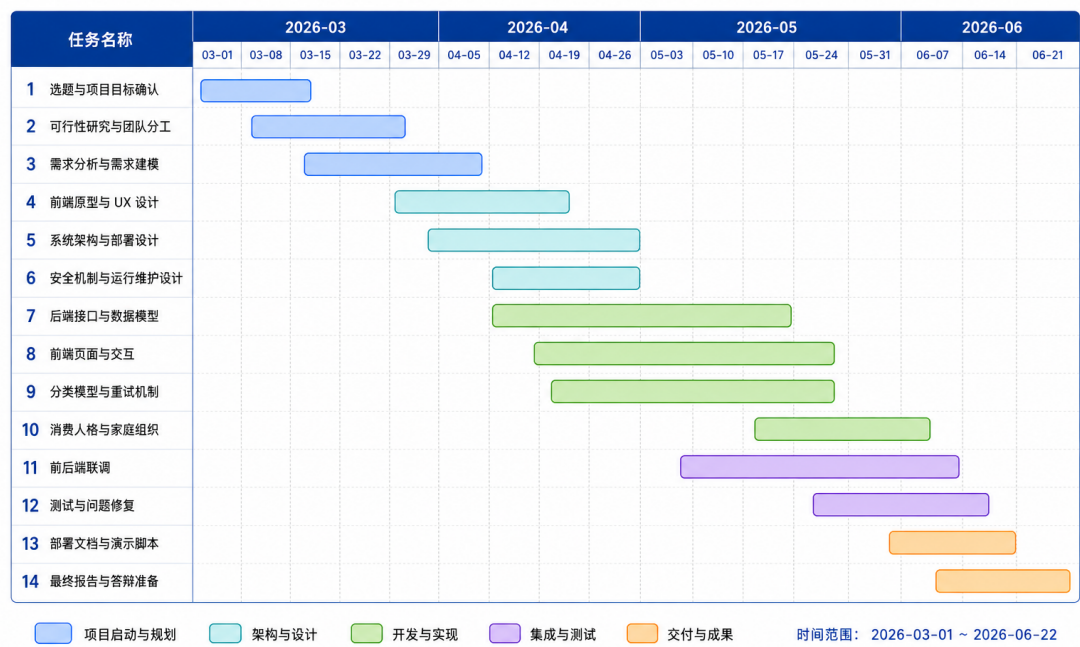


图 2.2 项目进度甘特图

2.4 团队组织与协作

2.4.1 团队成员与分工

项目由四名成员协作完成，各成员依据技术专长承担主要职责，并通过文档和规范降低人员交接成本。成员分工如表 2.6 所示。

表 2.6 团队成员与分工

成员	主要职责	备份与协作机制
郑博文	项目管理、需求统筹、答辩材料整理	维护进度计划和课程要求对照表，协调文档合并与版本管理。
林子尧	后端开发、数据库设计、接口实现	通过接口文档和自动化测试用例降低后端代码的交接成本。
单彬	前端页面、交互流程、图表展示	通过组件拆分和统一的 API 客户端封装降低前端页面的维护成本。
沈柔希	分类模型、测试、部署与联调	维护分类策略配置、测试流程和部署脚本，确保模型链路和部署方案可复现。

2.4.2 人员变动预案

为应对开发过程中可能出现的人员变动风险，团队制定了表 2.7 所示的应急预案。

表 2.7 人员变动应急预案

风险场景	应对方式
某成员生病或临时缺席	通过 Git 提交记录、接口文档和任务拆分让其他成员接手其负责模块的小范围修复工作。
某模块开发严重滞后	优先保证 MVP 核心闭环可演示，暂缓模块内的增强功能，待核心流程稳定后再补充。
答辩前关键资料缺失	以 README、项目文档、汇报 PPT 和测试结果作为可追溯的证据链，确保交付物完整性。

2.4.3 协作机制

团队协作依托以下工程实践：

- **版本控制**：使用 Git 进行代码版本管理，所有代码变更通过 Pull Request 合并，保持主分支可运行。
- **接口约定**：前后端以 OpenAPI 文档为契约，后端接口变更需同步更新文档，前端基于接口文档独立开发。

- **每日同步**：通过即时通讯工具保持进度同步，关键设计决策通过集中讨论确定并记录在项目文档中。
- **文档持续化**：需求、设计、测试和部署等各阶段文档随开发过程持续撰写和更新，避免期末集中整理。

2.5 风险管理

项目启动阶段对主要风险进行了识别和评估，并制定了相应的缓解措施。风险矩阵如表 2.8 所示。

表 2.8 项目风险矩阵

风险	概率	影响	等级	缓解措施
微信、支付宝账单格式差异	高	中	高	使用 ParserRegistry 模式处理多平台格式，所有解析器输出统一交易结构，保留测试样例。
大模型分类结果不稳定	中	中	中	在交易详情中展示分类理由和置信度；保留规则分类兜底和分类缓存；低置信度交易进入人工校正队列。
外部模型 API 不可用或延迟高	中	中	中	支持本地模型服务；外部请求通过统一重试队列串行处理，超时耗尽后标记为 retry_failed；可随时切回规则分类。
家庭组织权限配置错误	中	高	高	使用 OrganizationMember 角色校验 (owner/admin/member)；个人账本默认不受组织功能影响；所有组织接口校验成员身份。
后期时间不足	中	高	高	坚持 MVP 优先策略，先保证核心闭环可运行，再补充增强功能和文档润色。
多人协作接口不一致	中	中	中	以 OpenAPI 文档为前后端契约；后端测试覆盖核心接口；使用类型定义和统一 API 客户端约束前端请求。
安全设计不足	中	高	高	使用 JWT 鉴权、passlib 密码哈希、多用户 user_id 隔离；后续安全章节中列出已知改进项和加固建议。
大文件导入性能不足	中	中	中	当前限制上传文件规模；后续可通过异步任务、批量写入和 SQL 聚合优化大批量数据处理性能。

2.5.1 变更管理策略

在迭代开发过程中，各类变更按照表 2.9 所示的策略进行处理，以确保变更不会冲击核心闭环的稳定性。

表 2.9 变更管理策略

变更类型	处理方式
新增功能需求	先判断是否影响 MVP 核心闭环；若不影响则放入后续增强列表，优先保证当前阶段交付。
接口字段变化	后端更新接口文档（自动生成的 OpenAPI schema），前端同步修改 API 客户端和类型定义。
家庭组织权限变化	先保证个人账本默认行为不变，在此基础上扩展 organization_id 聚合查询的权限校验逻辑。
分类策略变化	保留规则兜底，记录每次分类的 provider、置信度和分类理由，确保分类链路可追溯。
页面交互变化	优先不破坏主流程（导入 → 分类 → 分析 → 报表），避免在交付前进行大范围 UI 改版。
部署环境变化	同时维护本地直接启动和 Docker Compose 两套方案，保障在任何环境下系统均可演示。

第三章 需求分析

3.1 需求获取方法

本系统的需求通过五种方法获取，覆盖从用户行为分析到工程质量审查的完整链路。

真实账单处理流程分析：项目启动阶段，团队收集支付宝与微信支付的原始账单文件，模拟用户从下载账单、整理数据到生成报表的完整操作流程，提炼出导入、解析、分类、分析、报表五大核心环节，作为功能需求的主要依据。

小组讨论与范围界定：团队依据开发周期与技术能力，对需求进行优先级排序，将核心闭环（认证、导入、分类、分析、报表）确定为最高优先级，将消费人格分析、家庭组织、模型切换等纳入扩展范围，形成清晰的 MVP 边界。

误分类案例分析：团队收集了多种语义误判场景——如“东南大学饭卡充值”被通用分类归为“学习”、连锁便利店消费被归为“购物”而非“餐饮”——驱动了大模型初分、分类理由展示和人工校正工作台的需求。系统需在给出分类建议的同时，提供置信度与推理理由，允许用户在校正工作台中批量修正。

原型与联调反馈：在前后端迭代过程中，前端页面的交互细节和后端接口的返回结构经过多轮联调调整。例如，导入页面的进度轮询机制、校正工作台的左右分栏布局、家庭组织的角色权限控制，均来自原型验证阶段的反馈。

工程质量审查：团队对照安全、隐私、性能、可部署性、可维护性、可测试性等质量维度，识别并补齐非功能需求。这一步骤确保系统不仅是功能可用的原型，也是具备工程质量的交付物。

3.2 利益相关者分析

系统涉及四类利益相关者，其关注点与需求来源如表所示。

表 3.1 利益相关者分析

利益相关者	关注点	需求来源
普通用户	上传账单、查看 AI 分类结果、修正语义误判、查看消费画像、生成报表	个人记账与财务分析场景
家庭组织管理员	创建家庭组织、添加成员、管理角色、查看家庭聚合分析	家庭账本协作场景
系统运维人员	配置模型供应商、管理重试队列、部署与监控系统	系统部署与运维场景
开发与维护人员	实现系统功能、控制开发范围、维护代码质量、准备后续扩展	工程开发与维护过程

3.3 功能需求

系统共识别 15 项功能需求，按优先级分为 P0（核心闭环必备）、P1（完整性与体验增强）和 P2（未来改进）。当前实现覆盖全部 P0 与 P1 需求。

表 3.2 功能需求列表

ID	需求描述	优先级	实现依据
FR-01	用户注册、登录及 Token 鉴权	P0	routes_auth.py、前端登录页、JWT 中间件
FR-02	多用户数据隔离，账单与报表按用户归属	P0	dependencies.py 依赖注入、资源归属校验
FR-03	支持微信/支付宝账单文件上传导入	P0	routes_imports.py、导入页面
FR-04	不同平台账单统一解析为标准化交易结构	P0	services/parsers.py、test_parsers.py
FR-05	大模型语义分类，记录来源、置信度与理由	P0	services/classifiers.py、test_classifiers.py

接下页

表 3.2 功能需求列表（续）

ID	需求描述	优先级	实现依据
FR-06	交易列表筛选（日期、平台、分类、关键词）与分页	P0	routes_transactions.py、交易列表页面
FR-07	人工校正低置信度或语义误判交易	P0	routes_transactions.py PATCH 接口、校正工作台
FR-08	用户自定义分类管理	P1	routes_categories.py、分类管理页面
FR-09	Dashboard 展示收入、支出、结余、储蓄率、分类占比	P0	routes_dashboard.py、services/analytics.py
FR-10	消费人格画像、财务健康评分与问卷自评	P1	routes_personality.py、services/personality.py
FR-11	家庭组织创建与成员角色管理	P1	routes_organizations.py、services/organizations.py
FR-12	家庭组织 Dashboard 与消费人格聚合	P1	organization_id 查询参数、组织页面
FR-13	PDF 报表生成与下载，支持个人与家庭模式	P0	routes_reports.py、services/reports.py
FR-14	分类供应商切换（规则/外部 API/本地模型）	P1	services/classifiers.py CompositeClassifier
FR-15	外部模型失败重试队列与管理员控制	P1	routes_classification.py、retry_queue.py

FR-01 至 FR-07、FR-09、FR-13 构成系统的核心闭环：认证后导入账单并自动分类，经人工校对后产出 Dashboard 分析与 PDF 报表。FR-08、FR-10 至 FR-12、FR-14、FR-15 在此闭环上增加了消费人格画像、家庭协作和运维可控性。

3.4 非功能需求

系统定义了 8 项非功能需求，涵盖安全性、隐私保护、可用性、性能、可部署性、可维护性、可测试性与可扩展性。

表 3.3 非功能需求列表

ID	需求名称	优先级	说明
NFR-01	安全性	P0	密码采用哈希存储，业务接口均需 JWT 鉴权，资源访问按用户归属校验
NFR-02	隐私保护	P0	账单数据、分类结果与 PDF 报表属于敏感个人财务数据，用户间严格隔离
NFR-03	可用性	P0	用户可按“导入 → 初分 → 校正 → 分析 → 报表”路径完成核心任务；家庭功能不破坏个人模式
NFR-04	性能	P1	小规模试运行场景下页面与接口保持可交互，大文件与模型调用有风险控制
NFR-05	可部署性	P0	支持本地开发模式、Docker Compose 部署与裸机部署三种方式
NFR-06	可维护性	P0	前端后端分离；后端按 routes/services/models/schemas 分层组织
NFR-07	可测试性	P0	后端具备 pytest 自动化测试套件，前端通过 TypeScript 类型检查与生产构建验证
NFR-08	可扩展性	P1	分类供应商、报表生成、异步任务与部署组件预留扩展接口

3.5 用户场景分析

3.5.1 普通用户场景

普通用户的核心场景覆盖“导入—分类—校正—分析—报表”的完整链路：

1. 用户注册账号并登录系统，进入应用主界面。
2. 进入账单导入页面，上传微信或支付宝账单文件（CSV 格式）。
3. 系统创建导入批次，后台解析账单并将不同平台格式转换为统一交易记录。

4. 系统调用大模型对每条交易进行语义初分，记录分类来源、置信度与分类理由。
5. 用户在交易列表中按日期、平台、分类、关键词筛选查看交易。
6. 在分类校正工作台，用户查看待校正交易队列，对低置信度或语义误判的分类进行人工修正。
7. 用户进入 Dashboard 查看收入支出概览、储蓄率、分类占比图与 Top 商户。
8. 用户进入消费人格页面，查看基于交易数据计算的消费人格画像、财务健康评分，并可通过问卷进行自评对比。
9. 用户进入报表中心，选择日期范围与导入文件，生成并下载 PDF 报表。

3.5.2 家庭组织场景

家庭组织场景描述管理员创建家庭、管理成员并查看聚合分析的过程：

1. 用户创建家庭组织，系统自动将创建者设为 owner 角色。
2. owner 或 admin 通过用户名搜索添加成员，并可提升普通成员为 admin。
3. 家庭成员进入组织页面查看家庭合计收入、支出、交易数与消费人格聚合雷达图。
4. owner 或 admin 可触发家庭范围内的重分类操作；普通 member 只能查看聚合结果。
5. 不传入 organization_id 参数时，Dashboard、Personality 和 Reports 仍按个人账本模式工作，保持向后兼容。

3.5.3 系统运维场景

运维人员关注系统部署、配置与故障恢复：

1. 运维人员配置后端运行环境，包括数据库连接、Redis 缓存、模型供应商与 API 密钥。
2. 部署前端静态资源、Nginx 反向代理、FastAPI 后端、PostgreSQL 与 Redis。
3. 当模型请求超时或失败时，运维人员通过重试状态接口查看队列中的交易数量与供应商分布。
4. 运维人员通过管理界面将历史超时或失败交易重新放入重试队列，恢复分类流程。

3.6 异常场景处理

系统对以下异常场景定义了明确的预期行为：

表 3.4 异常场景与系统预期行为

异常场景	系统预期行为
用户未登录访问业务接口	返回 401 未授权状态码，前端跳转至登录页
用户上传不支持的账单格式	导入失败并记录具体错误信息，页面提示用户重新上传
重复导入同一笔交易	通过 dedupe_hash 字段去重，避免重复记录入库
模型供应商不可用	交易进入重试队列，后台串行重试；超时耗尽后标记为 retry_failed
用户尝试访问他人报表或交易	后端按 current_user.id 校验资源归属，拒绝非授权访问
报表生成失败	记录任务状态与错误信息，用户可调整参数后重新触发生成

3.7 用例建模

系统用例图展示三类参与者（普通用户、家庭管理员、系统运维人员）与系统功能之间的关系。普通用户可操作认证、导入、查看交易、校正分类、管理自定义分类、查看 Dashboard、生成报表、查看消费人格等用例；家庭管理员额外可创建组织和管理成员；系统运维人员负责供应商配置与重试队列管理。导入账单用例包含大模型语义分类作为扩展关系。

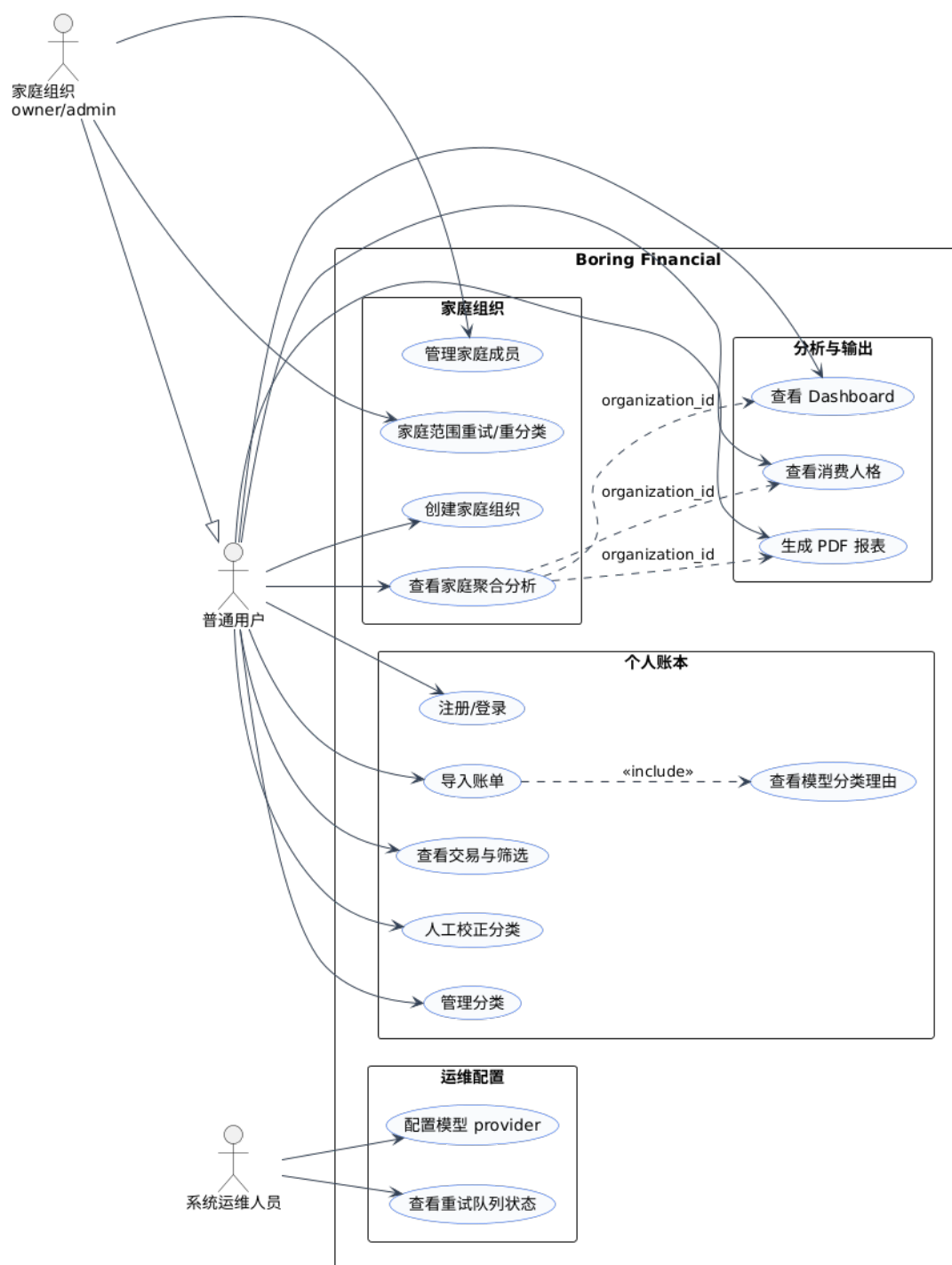


图 3.1 系统用例图

3.8 业务流程建模

核心业务活动图描述了从用户登录到报表生成的端到端流程。流程始于用户登录，接着上传账单文件，系统创建导入批次后进行解析和去重；成功解析的交易经过大模型语义初分，高置信度结果直接入库，低置信度交易进入人工复核队列；用户完成校正后，交易数据进入 Dashboard 聚合分析，随后流向消费人格与财务健康分析模块；在报表生成环节，系统根据是否选择家庭组织参数，分别生成个人 PDF 报表或家庭聚合 PDF 报表。

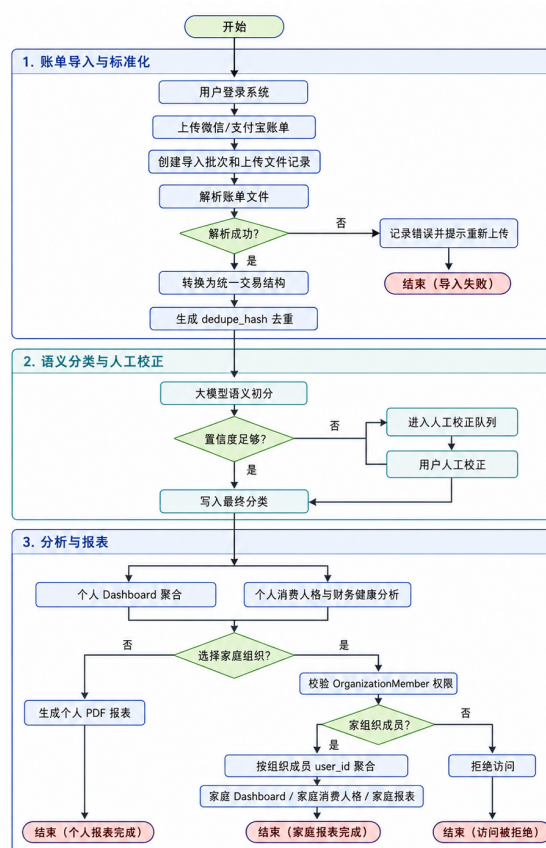


图 3.2 核心业务活动图

第四章 用户界面与体验设计

4.1 设计系统

系统采用专业金融工具风格的设计体系，以深色导航与浅色内容区形成清晰的视觉层级。设计系统的核心参数如下：

表 4.1 设计系统色板

色板角色	色值	应用位置
深色侧边栏底色	#06233D	左侧导航栏背景
深色侧边栏悬浮色	#041A2E	导航项悬停/选中态
品牌主色	#00A884	主要按钮、链接、数据高亮
页面背景	#F5F7FA	内容区域整体背景
卡片背景	#FFFFFF	数据卡片、表格、表单容器
成功色	#22C55E	成功状态、收入标识
警告色	#F59E0B	警告提示、中等置信度
错误色	#EF4444	错误状态、支出标识
信息色	#3B82F6	信息提示、链接、进度条

深色侧边栏提供全局导航，顶部栏显示当前页面标题与用户信息区域。主内容区采用白色卡片承载数据面板、表格与表单，通过规则间距与统一圆角保持视觉一致性。图表采用 ECharts 渲染，配色与设计系统色板保持一致，确保图表与页面风格协调。

4.2 主要页面原型

系统共设计 10 个主要页面，本节逐一描述各页面的功能定位与关键交互。

4.2.1 登录与注册页面

登录与注册页面采用左右分栏布局：左侧为深色品牌展示区，包含应用名称与简短说明；右侧为白色表单卡片，包含用户名与密码输入框，以及登录与注册模式切换按钮。注册模式额外显示邮箱字段。验证成功后，系统存储 JWT Token 并跳转至 Dashboard。

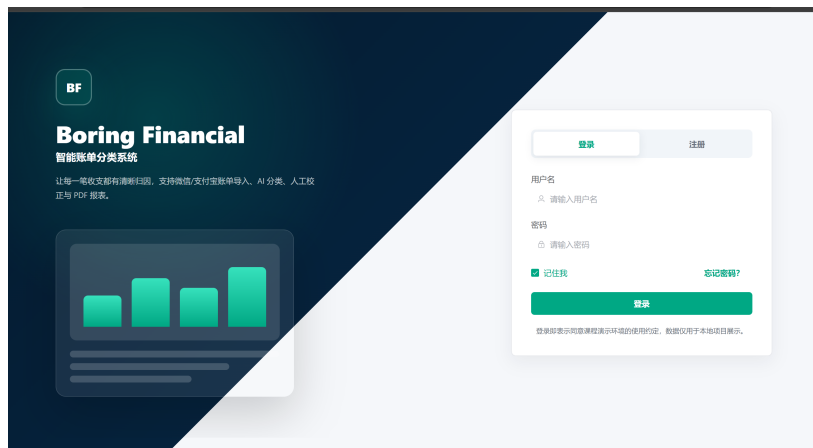


图 4.1 登录与注册页面截图

4.2.2 仪表盘页面

Dashboard 是系统的财务驾驶舱，顶部展示收入、支出、结余、储蓄率四张摘要卡片。中部为支出趋势折线图和分类占比饼图，分别使用 ECharts 渲染。右侧展示 Top 商户排行榜。页面数据通过 `/api/dashboard/summary` 接口聚合生成，用户可通过顶部日期筛选器切换统计周期。

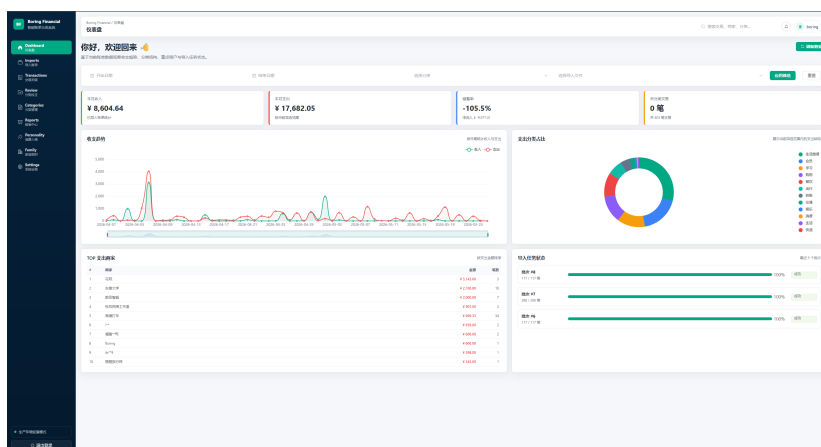


图 4.2 仪表盘页面截图

4.2.3 账单导入页面

账单导入页面提供拖拽上传区域，支持微信与支付宝 CSV 账单文件。上传成功后，页面展示最近导入记录列表，包含文件名、平台、状态与处理进度条，支持批量进度轮询。用户可点击导入记录查看详情，或删除历史导入批次。

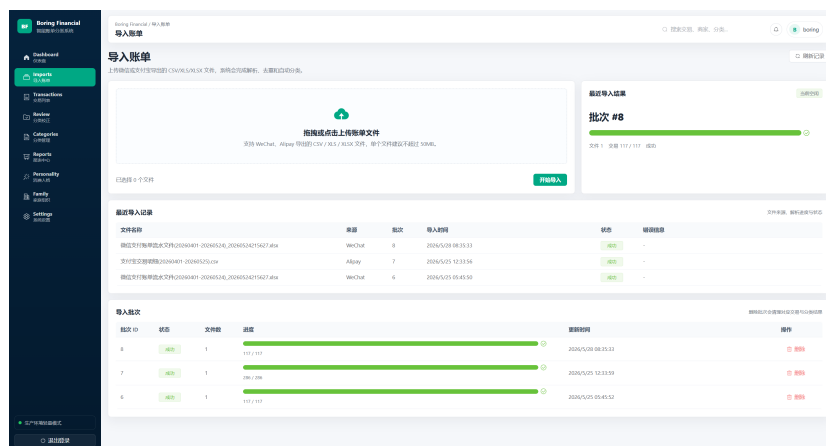


图 4.3 账单导入页面截图

4.2.4 交易列表页面

交易列表页面以筛选栏加分页表格的形式展示所有交易记录。筛选条件包括日期范围、平台（微信/支付宝）、分类、导入文件和关键词搜索。表格每行显示时间、商户、金额、分类与置信度。点击行可展开详情抽屉，展示分类来源、置信度、分类理由与原始交易备注。

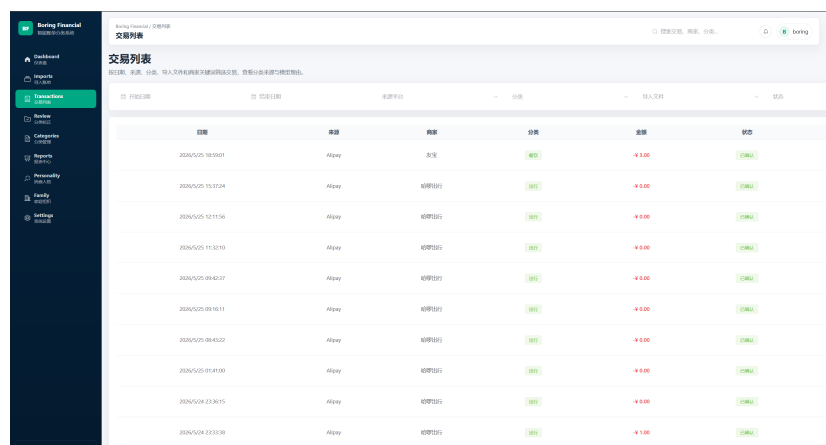


图 4.4 交易列表页面截图

4.2.5 分类校正工作台

校正工作台采用左右分栏布局：左侧为待校正交易队列，按置信度从低到高排序，每项显示商户、金额与当前分类；右侧为当前选中交易的详细信息，包括模型分类建议（来源、置信度、推理理由），以及分类选择控件，允许用户选择新的系统分类或自定义分类并确认修正。确认后交易从队列中移除。

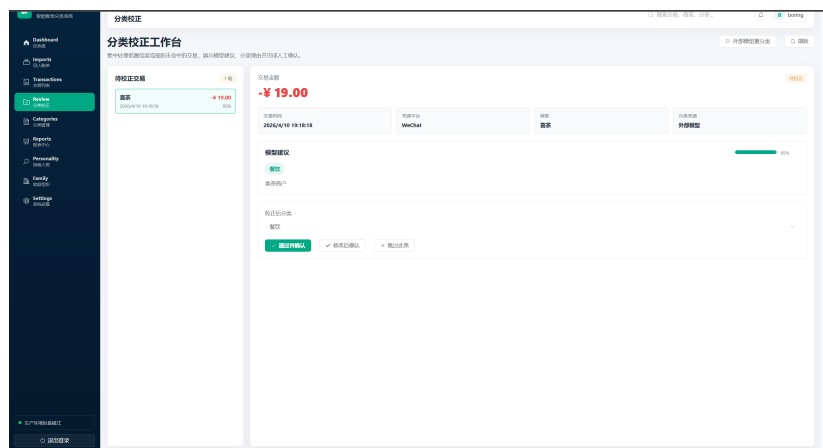


图 4.5 分类校正工作台截图

4.2.6 分类管理页面

分类管理页面展示系统预设分类与用户自定义分类。顶部为分类使用频率统计，按交易数量排序。中部为分类列表，系统分类标记为锁定不可删除，用户自定义分类支持新增、编辑与禁用操作。修改操作通过弹窗表单完成，实时生效。

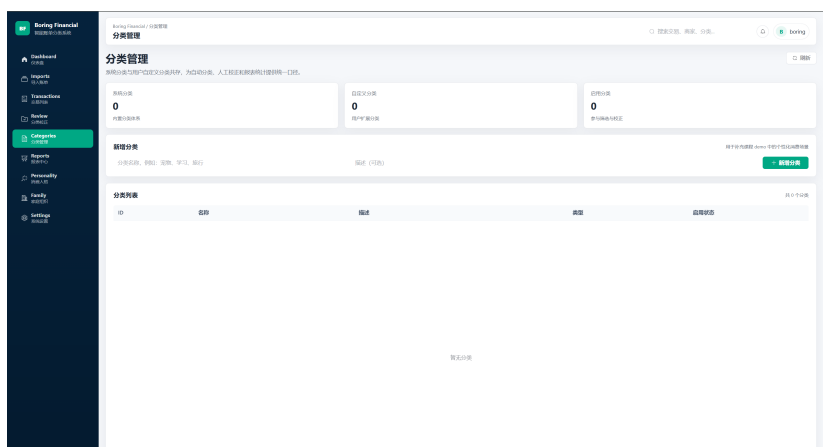


图 4.6 分类管理页面截图

4.2.7 消费人格页面

消费人格页面由三个区域组成：顶部为 16 型消费人格画像卡片，展示用户所属人格类型及简要描述；中部为四维雷达图（现时偏好、心理账户、炫耀性消费、开放性），展示各维度得分；底部为财务健康评分区域，展示储蓄率、收入稳定性、消费多样性、支出波动性和应急能力的子指标。页面同时提供问卷自评入口，用户可对比主观认知与交易数据画像的偏差。

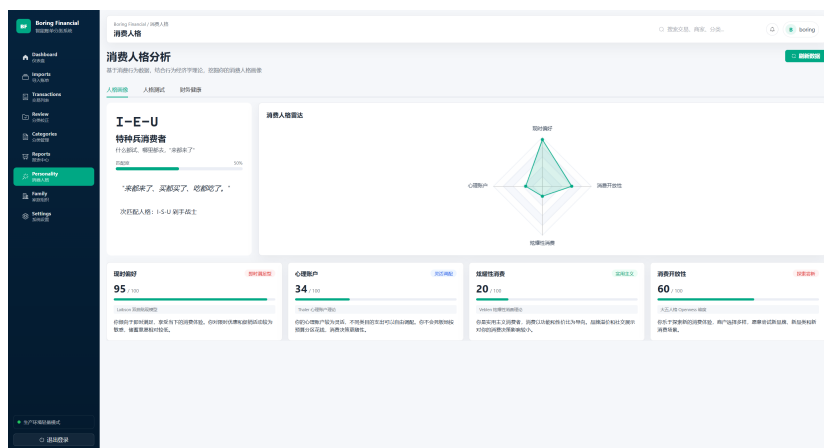


图 4.7 消费人格页面截图

4.2.8 家庭组织页面

家庭组织页面在用户未加入任何组织时显示创建引导。已有组织时，页面展示三部分内容：左侧为成员列表，每行显示用户名、角色（owner/admin/member）与角色管理按钮；右上方为家庭财务概览卡片，展示组织合计收入、支出与交易数；右下方为家庭消费人格聚合雷达图，将所有成员交易数据按人格维度聚合展示。owner 和 admin 可见角色管理控件与重分类触发按钮。

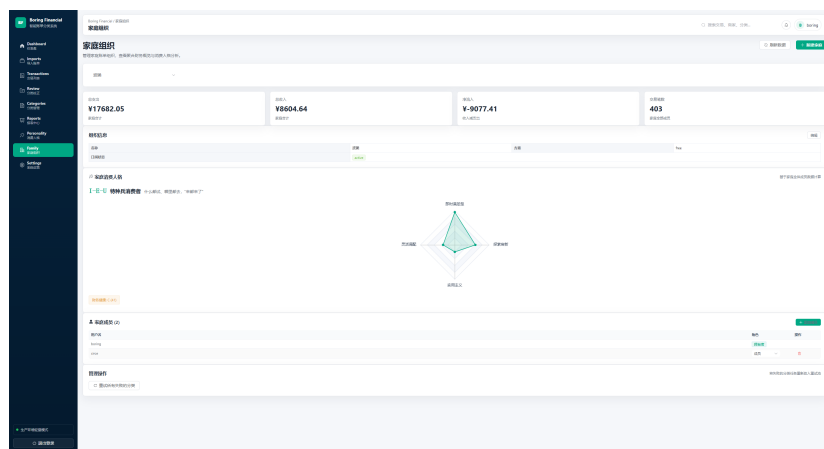


图 4.8 家庭组织页面截图

4.2.9 报表中心页面

报表中心页面提供日期范围选择器与导入文件筛选器。用户设置条件后点击生成按钮，系统异步创建报表任务。生成完成后，页面展示报表摘要信息（覆盖时间段、交易数、分类分布），并提供 PDF 下载按钮。历史报表列表按生成时间倒序排列，支持重新下载。

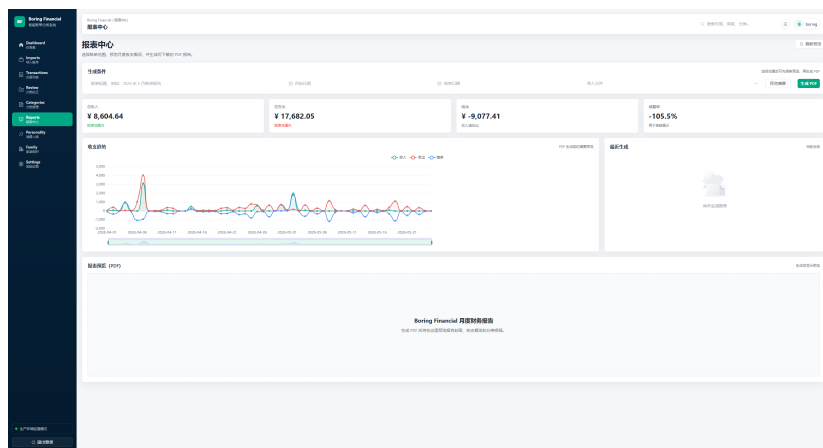


图 4.9 报表中心页面截图

4.2.10 系统设置页面

系统设置页面向所有用户展示当前分类供应商名称、置信度阈值等配置信息。管理员用户额外可见重试队列控制区：包括当前队列状态（等待重试数、重试中数、已失败数），以及“一键重试历史超时”按钮。普通用户仅可见配置展示，管理操作控件隐藏。

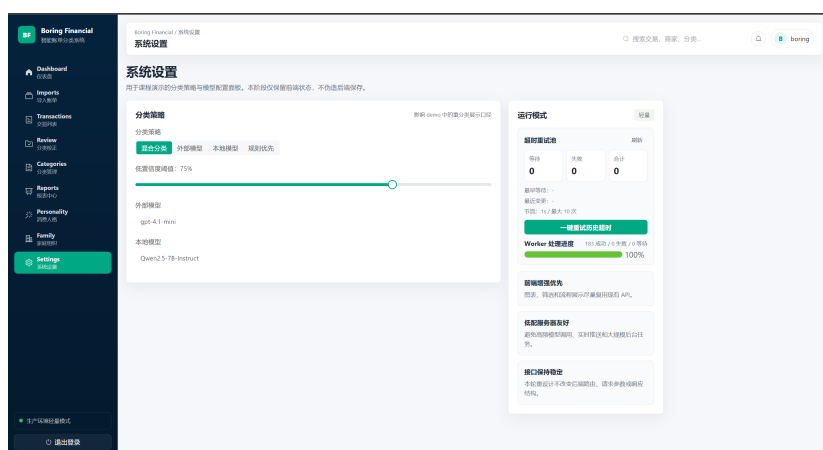


图 4.10 系统设置页面截图

4.3 前端架构与组件职责

前端基于 Vue 3 + TypeScript + Element Plus + ECharts 技术栈，采用组件化分层架构。各核心模块的职责划分如表所示。

表 4.2 前端核心组件与模块职责

模块/文件	职责
layouts/AppLayout.vue	整体布局外壳：深色侧边栏、顶部标题栏、用户信息区与主内容渲染区域
pages/ (10 个页面组件)	各业务页面的视图逻辑与交互，对应登录、Dashboard、导入、交易、校正、分类、人格、组织、报表、设置
stores/auth.ts	认证状态管理：用户名、Token 存取、登录状态持久化至 localStorage
api/client.ts	Axios 实例封装：统一注入 Bearer Token、请求/响应拦截、错误处理
router/index.ts	前端路由定义：页面懒加载、路由守卫（未登录重定向至登录页）
types/	TypeScript 接口定义：交易、分类、导入批次、报表、组织成员等数据结构的类型约束

前端通过 Axios 拦截器在每次请求中自动附加 Authorization 头部，确保所有 API 调用均携带有效 JWT Token。路由守卫在页面切换时检查认证状态，未登录用户自动重定向至登录页。页面组件通过 Pinia Store 获取全局状态（认证信息、当前组织），本地状态（筛选条件、分页）在组件内部管理。

4.4 用户体验目标

系统用户体验设计围绕以下四个核心目标展开：

减少用户操作步骤：账单导入采用拖拽上传模式，一次上传即可触发解析 → 去重 → 分类的全自动链路。Dashboard 与人格页面的数据通过后端聚合计算，用户无需手动维护分类映射表或公式。批量校正工作台允许用户在单一页面完成所有低置信度交易的复核，无需逐条跳转。

分类结果可解释、可校正：模型分类结果不仅给出类别，还提供置信度与人类可读的推理理由。校正工作台将模型建议与修改控件并置，用户可在了解 AI 推理过程后做出校正决策。校正结果即时生效，无需额外保存步骤。

校正数据全链路贯通：人工校正后的交易分类应实时反映在 Dashboard 聚合、消

费人格计算和 PDF 报表中。系统通过统一数据源（transactions 表）确保各分析模块使用一致的最新分类结果，避免出现“已校正但图表未更新”的不一致情况。

个人模式不受家庭功能影响：家庭组织功能采用显式 opt-in 设计——用户需主动创建或加入组织，并在 Dashboard 和报表页面通过 `organization_id` 参数显式切换至家庭视图。不传入该参数时，所有页面保持原有个人模式行为，确保现有用户的体验不受影响。

第五章 系统设计

本章阐述 Boring Financial 智能账单分类系统的架构设计，涵盖总体架构、技术选型、前后端分层设计、数据隔离策略、部署方案以及安全机制。

5.1 总体架构

系统采用四层架构，自上而下分为展示层、网关层、服务层与数据/外部服务层，各层通过 HTTP 协议通信，职责边界清晰。

1. **展示层**: Vue 3 + Element Plus + ECharts 构建的单页应用，运行于用户浏览器，负责全部用户交互与数据可视化。
2. **网关层**: 生产环境使用 Nginx 对外提供静态文件服务，并通过反向代理将 /api 请求转发至后端。开发环境使用 Vite Proxy 实现同源转发。
3. **服务层**: FastAPI 构建的 RESTful API 服务，包含认证、导入、交易、分类、Dashboard 聚合、消费人格、家庭组织及 PDF 报表等业务模块。异步任务通过 Celery Worker 消费，外部模型调用通过 Retry Queue Worker 后台线程串行管理。
4. **数据与外部服务层**: PostgreSQL (生产) / SQLite (轻量部署) 作为主数据库，Redis 作为 Celery 消息代理。外部服务包括 OpenAI-compatible API 与本地模型服务 (vLLM)，由 Retry Queue Worker 统一调度。

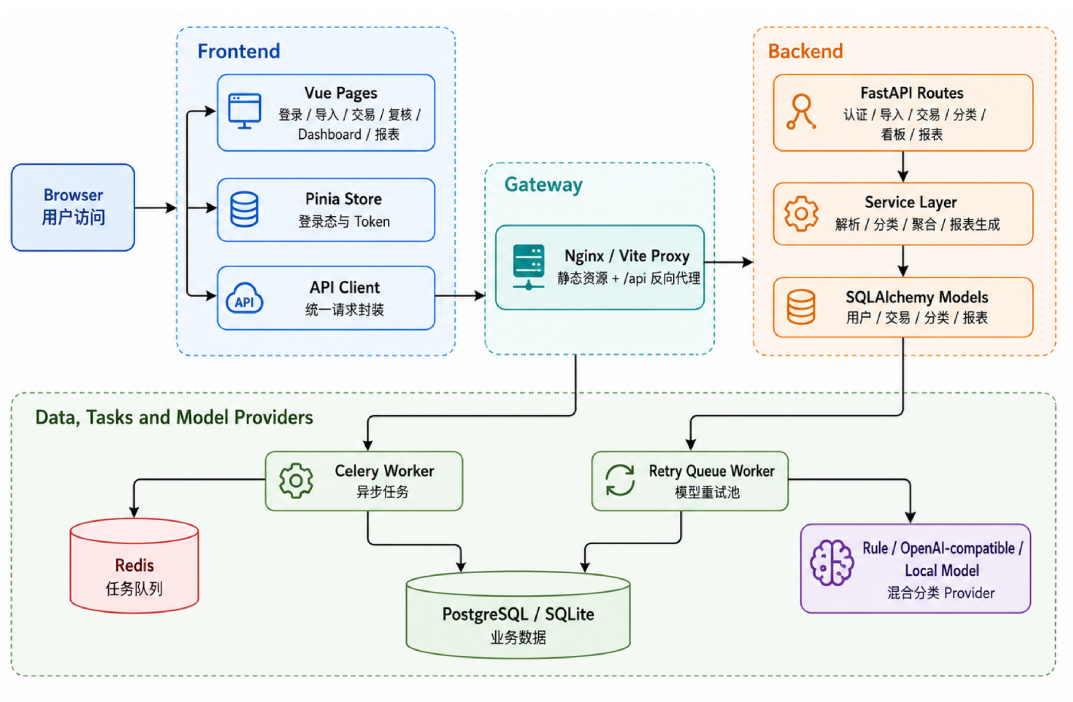


图 5.1 系统总体架构图

5.2 技术选型

下表列出各层采用的核心技术及其选型依据。

表 5.1 技术选型汇总

技术	用途	选型依据
FastAPI (Python)	后端 API 框架	原生异步支持，自动生成 OpenAPI 文档 (/docs、/redoc)，Pydantic 请求/响应验证
Vue 3 + Type-Script	前端框架	组合式 API 支持高复用的逻辑组织，Type-Script 编译期类型检查降低运行时错误
Element Plus	UI 组件库	成熟的 Vue 3 组件生态，提供表格、表单、弹窗、导航等企业级组件
ECharts	数据可视化	支持雷达图、柱状图、折线图、饼图等，满足 Dashboard 与人格分析的可视化需求
SQLAlchemy 2.x	ORM 框架	Data Mapper 模式解耦领域逻辑与持久化，支持 SQLite/PostgreSQL 双数据库切换
Celery + Redis	异步任务队列	Celery 提供生产级任务调度与重试，Redis 作为轻量消息代理兼缓存
fpdf2	PDF 生成	轻量级 Python PDF 库，无外部依赖，支持 Unicode 中文字体加载
Docker Compose	容器编排	一键启动多服务栈，确保开发/测试/生产环境一致性

5.3 前端架构

前端代码按职责划分为四层：视图层由 10 个页面组件构成，对应登录、Dashboard、导入、交易列表、校正工作台、分类管理、消费人格、家庭组织、报表中心和系统设置；状态层通过 Pinia Store 管理认证状态与 Token 持久化；通信层基于 Axios 封装 API 客户端，通过请求拦截器自动注入 Bearer Token；路由层使用 Vue Router 配置页面懒加载与导航守卫，未登录用户自动重定向至登录页。类型定义层为全部业务模型提供 TypeScript 接口约束。

前端视觉设计规范以深色导航（#06233D）、品牌色（#00A884）与浅灰页面背景（#F5F7FA）构成统一视觉语言，详见第 4 章。

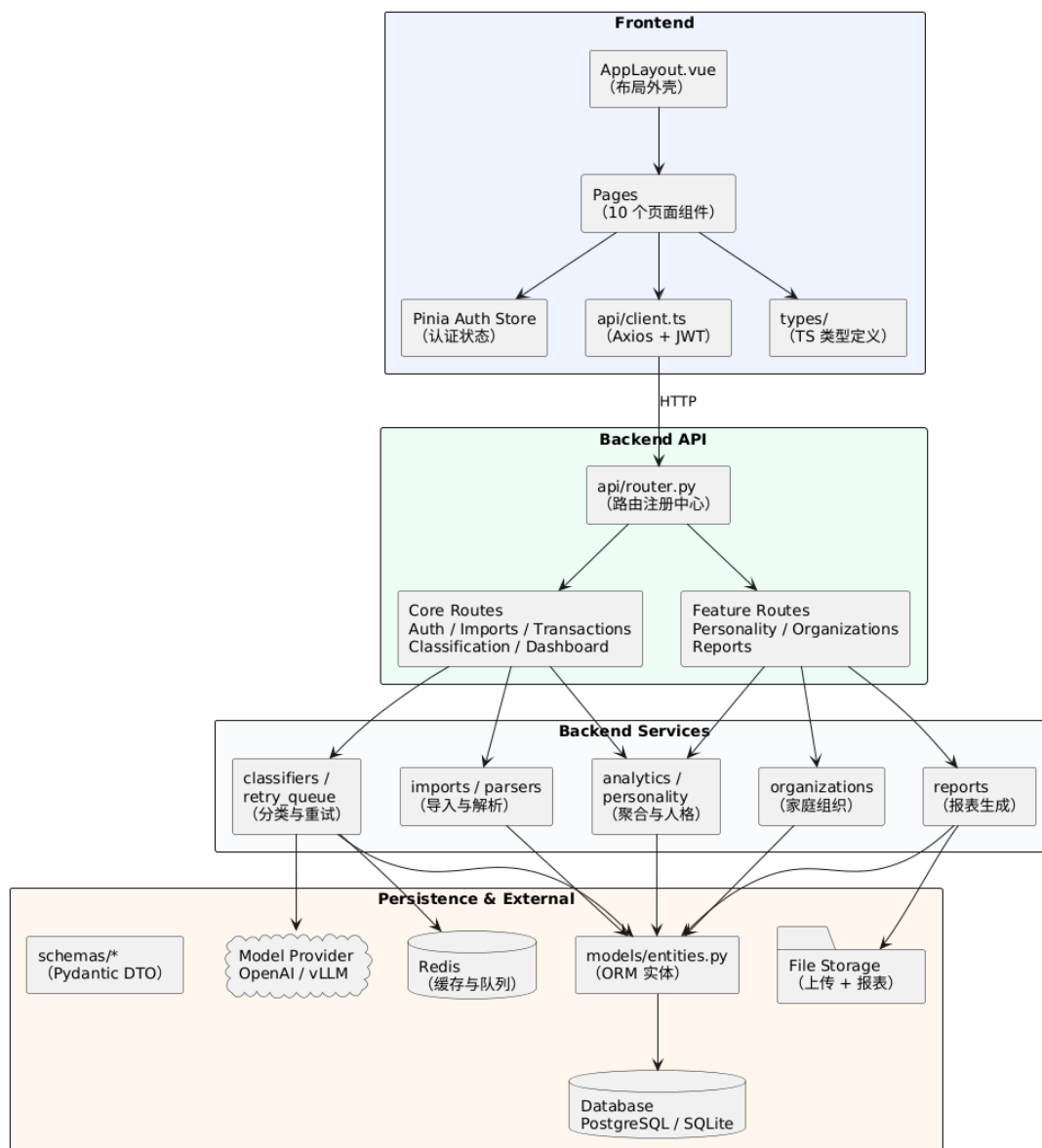


图 5.2 UML 组件图

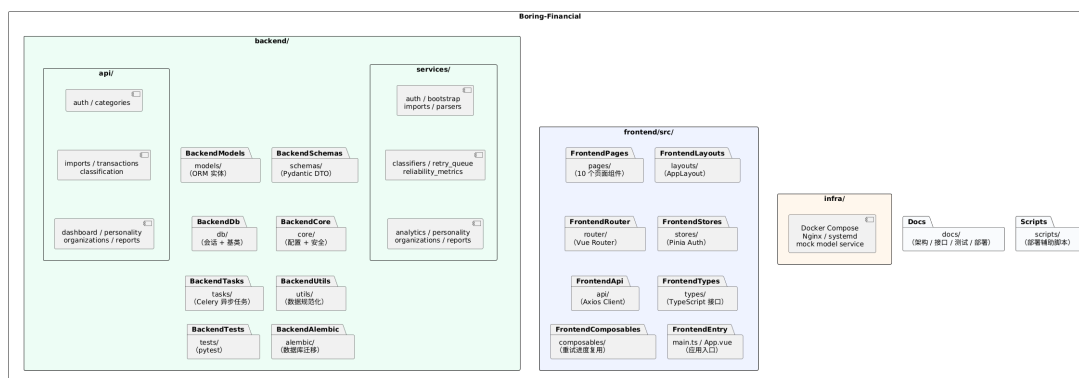


图 5.3 系统包图

5.4 后端架构

后端按分层架构组织为四层。路由层 (api/) 通过 router.py 统一注册 10 个路由模块, 覆盖认证、分类、导入、交易、分类校正、Dashboard、人格分析、家庭组织和报表生成。服务层 (services/) 承载全部业务逻辑: imports.py 管理导入批次与文件处理, parsers.py 实现多平台账单解析与规范化, classifiers.py 以复合分类链路编排规则、缓存与模型调用, retry_queue.py 提供后台串行重试池, analytics.py 与 personality.py 负责聚合计算与人格分析, organizations.py 实现家庭组织 CRUD 与角色校验, reports.py 以 Builder 模式构建 PDF 报表。数据层(models/entities.py) 定义 12 个 SQLAlchemy ORM 实体, schemas/ 按业务域拆分的 Pydantic DTO 负责请求校验与响应序列化。基础设施层(db/、core/、tasks/) 提供数据库会话管理、JWT 安全模块、配置加载与 Celery 异步任务定义。

5.5 数据隔离策略

系统通过以下机制保障多用户数据隔离与家庭组织聚合访问:

1. **个人资源归属**: 所有用户私有资源 (交易、导入批次、上传文件、分类缓存、报表等) 通过 user_id 外键归属。系统分类的 user_id 为 NULL, 用户自定义分类的 user_id 指向创建者。
2. **查询过滤**: 业务接口通过 Bearer Token 提取当前用户身份, 查询时强制附加 WHERE user_id = current_user.id 条件。
3. **家庭聚合模式**: 家庭组织不改变交易归属。用户选择家庭视图时, 前端传入可选 organization_id。后端先通过 OrganizationService 校验成员身份, 再收集组织内全部成员 user_id 列表, 传入 Dashboard、Personality 或 Reports 服务实现跨成员数据聚合。
4. **默认模式**: organization_id 为可选参数, 不传入时各接口按个人账本口径返回数据, 保持向后兼容。

5.6 部署架构

系统支持两种部署模式以适应不同规模的运行环境:

轻量本地部署 使用 SQLite 作为数据库, TASK_ALWAYS_EAGER = true 使 Celery 任务在当前进程同步执行, Retry Queue Worker 在后台线程运行。前端通过 Vite 开发服务器或构建后的静态文件直接访问。

生产环境部署 Docker Compose 编排多容器服务栈: PostgreSQL 数据库、Redis 消息代理、Nginx 反向代理 + 静态文件服务、FastAPI/Uvicorn (监听 127.0.0.1:8000)、Celery Worker、Retry Queue Worker 后台线程。外部模型服务作为独立外部依赖。

通信链路为：用户浏览器 → Nginx（静态资源直接返回，API 请求反向代理至 FastAPI）→ FastAPI 与 PostgreSQL、Redis、Retry Queue Worker 通过内部网络通信。上传文件与 PDF 报表存储于 storage/ 目录。

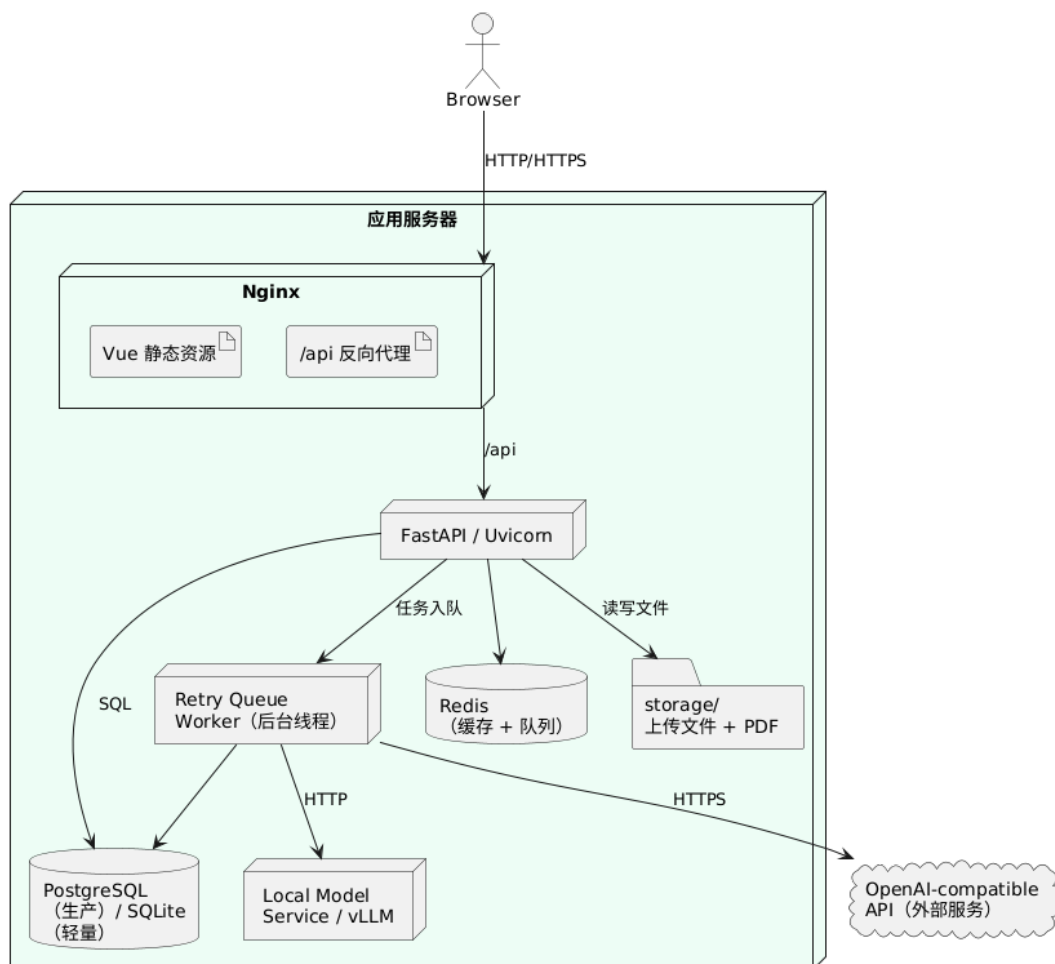


图 5.4 系统部署图

5.7 安全机制

系统的安全设计覆盖认证、授权、数据隔离、注入防护与隐私保护五个层面。

表 5.2 安全机制汇总

安全风险	应对措施	实现位置	说明
身份伪造	密码哈希 (passlib pbkdf2_sha256) , JWT 双令牌 (access + refresh)	core/security.py	密码不存明文; access token 短期有效
未授权访问	Bearer Token 强制验证用户身份	api/dependencies.py	FastAPI Depends 注入, 每条请求自动解码 JWT
跨用户数据泄露	私有资源查询强制附加 user_id 过滤	各 API 路由及服务层	SQLAlchemy ORM 参数化查询自动转义
组织越权操作	RBAC 角色模型 (owner/admin/member)	services/organization.py	owner 可管理成员与角色, member 仅可查看
SQL 注入	SQLAlchemy ORM 参数化查询	models/entities.py	不存在字符串拼接 SQL 的场景
恶意文件上传	文件大小限制、扩展名白名单 (CSV/XLSX)	routes_imports.py	后端验证文件类型
模型隐私	商户名称与商品摘要可被发送至外部 API	services/classifiers.py	识别为风险点; 缓解措施含本地模型选项

安全模块通过 FastAPI 的 Depends 机制将认证逻辑与业务代码解耦: get_current_user 从请求头提取并验证 JWT, require_admin 在此基础上进一步校验管理员身份。此设计使安全策略变更不影响业务逻辑。

第六章 程序开发

本章阐述系统的程序组织方式、数据库设计、设计模式应用以及核心流程设计。面向对象设计集中体现在四个子系统：领域实体模型、分类器策略链、账单解析器注册表与报表构建器。

6.1 面向对象程序组织

系统并非全量采用面向对象范式——在用户认证、API 路由、Dashboard 聚合等模块中，函数式与过程式组织更为自然。OO 原则集中于以下四个区域。Pydantic DTO 与 FastAPI 依赖注入在上一章安全机制部分已做阐述。

6.1.1 领域实体及其关联关系

系统使用 SQLAlchemy ORM 定义 12 个实体，统一继承 Base(DeclarativeBase)，采用 Data Mapper 模式将领域模型与持久化逻辑解耦——实体类不包含 SQL 语句，所有查询通过 Session 与 Select 构建器完成。

TimestampMixin (models/entities.py:12) 通过 Python 多重继承为 8 个实体提供 created_at/updated_at 字段，以 Mixin 模式横向复用时间戳关注点，消除字段声明重复。

实体关联的核心拓扑如下：User 位于关联中心，向多个方向辐射——通过 import_batches 管理导入任务，通过 transactions 持有全部交易记录，通过 categories 维护自定义分类；ImportBatch 聚合 UploadedFile，两者共同关联至 Transaction；Transaction 通过 auto_category_id 与 final_category_id 双 FK 指向 Category，同时向 ClassificationResult 和 ClassificationCache 辐射分类历史与缓存；家庭组织域中 Organization 通过 OrganizationMember 关联表与 User 建立多对多关系。

Transaction 表基于 (user_id, dedupe_hash) 唯一约束实现去重，dedupe_hash 由时间、金额、商户、商品、备注五字段的规范化值经 SHA-256 生成。双分类字段设计中 auto_category_id 存储模型/规则的建议分类(可被重分类更新)，final_category_id 存储人工修正后的最终分类，非 NULL 时优先于自动分类。ClassificationResult 表只增不改，每次分类操作新增一条记录，用于追溯分类器表现变化。

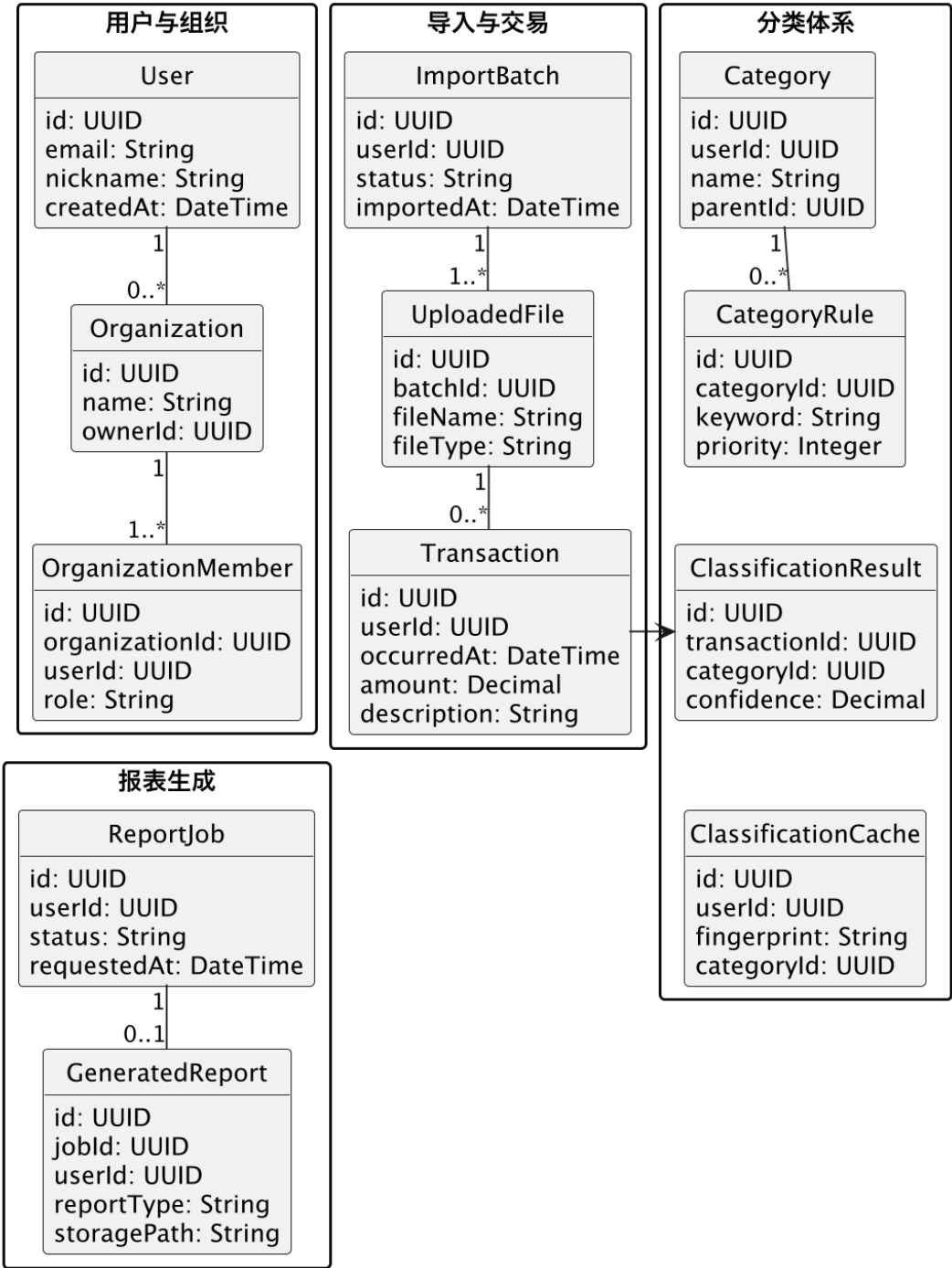


图 6.1 领域实体类图

6.1.2 分类器策略

分类子系统围绕策略、责任链与外观三种模式组织，由五个类构成。

`CompositeClassifier` (`services/classifiers.py:267`) 作为外观入口封装分类全流程。其 `classify()` 方法编排四级责任链：检查 `ClassificationCache` 缓存 → 调用外部模型 API → 调用本地模型 → `RuleBasedClassifier` 规则兜底。外部模型请求通过重试队列异步入队，避免并发 API 调用；超时异常经指数退避重试，达到上限后标记为 `retry_failed` 并从活跃队列移除。`OpenAICompatibleClassifier` 与 `LocalModelClassifier` 均实现统一的 `classify(transaction) -> ClassificationOutput` 调用约定——前者通过 `httpx` 调用 `OpenAI-compatible /chat/completions`，后者继承前者并覆盖 `provider` 配置指向本地 `vLLM` 服务。`RuleBasedClassifier` 按优先级匹配分类规则，命中返回置信度 0.99，未命中返回 `None`。`ClassificationOutput` 作为 `dataclass` 统一承载分类结果。

`Strategy` 模式使三种分类器可互换，各自封装不同的分类逻辑。`Chain of Responsibility` 提供优雅降级路径——缓存命中直接返回，API 不可用时自动降至规则。`Facade` 模式将缓存管理、重试逻辑、结果存储等复杂性隐藏在 `classify()` 内部，调用方只需传入交易对象。

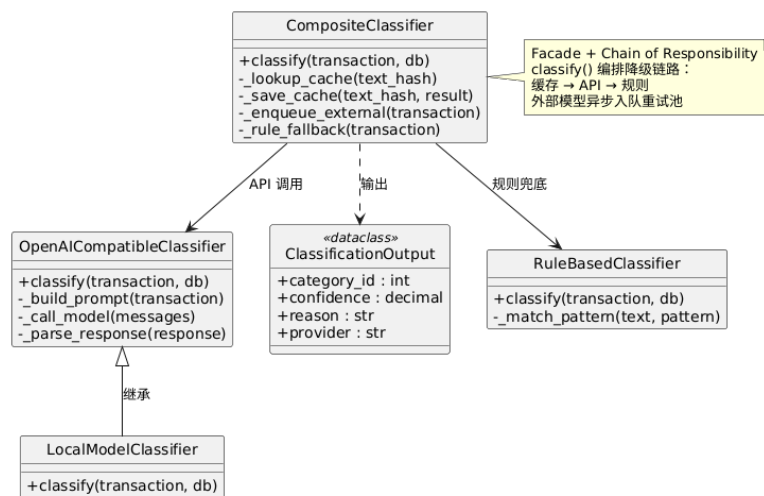


图 6.2 分类器策略类图

6.1.3 账单解析器

解析子系统负责将微信/支付宝异构账单文件转换为统一交易结构，采用策略、适配器与注册表三种模式。

`ParserRegistry` (`services/parsers.py:239`) 作为工厂入口，`parse_file(path)` 方法执行完整流程：通过文件名关键字与文件内容预览检测平台 → `StatementReader` 读取文件（自动检测 6 种编码、定位表头行、适配 Excel/CSV 格式）→ 选择对应平台解析器 → 转换为 `ParsedTransaction` 统一结构 → `TransactionNormalizer` 对商户、商品、备注字段做 NFKC 归一化并生成 SHA-256 去重哈希。

AlipayParser 与 WeChatParser 各自维护一套平台特有的列名映射（如支付宝的「交易对方」→ merchant、微信的「当前状态」→ status），将原生 DataFrame 列映射为 ParsedTransaction 统一结构。Strategy 模式使平台解析器可独立扩展——新增平台只需添加解析器并在 ParserRegistry._ordered_parsers() 中注册。Adapter 模式体现在 StatementReader 类——将 pandas 的复杂 API（多编码、多格式、多表头位置）适配为统一的 read(path) 接口。Registry/Factory 模式通过 ParserRegistry 实现解析器的自动选择与调度。

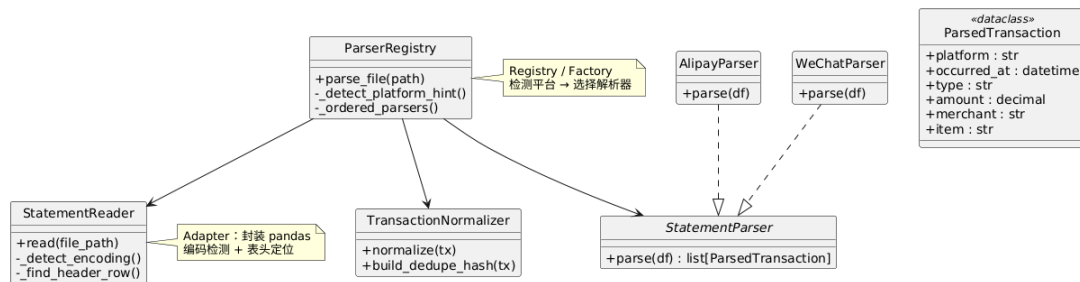


图 6.3 账单解析器类图

6.1.4 报表构建器

报表生成由 ReportBuilder(services/reports.py:16) 单一类实现，采用 Builder 模式将复杂 PDF 构建过程拆分为有序步骤。

build() 方法编排完整构建序列：_build_context() 聚合查询交易数据与分类映射 → _add_cover() 渲染深蓝色封面 → _section_title() 与 _metric_row() 渲染摘要指标卡片 → _simple_table() 渲染分类汇总与 Top 商户表格 → 逐分类渲染交易明细页 → 保存 PDF 并写入 GeneratedReport 记录。_resolve_unicode_font() 按优先级搜索 8 个候选路径（项目内置字体、Windows/Linux 系统字体目录），自动检测中文字体。

Builder 模式使每个方法关注单一渲染关注点，扩展新章节只需新增方法并在 build() 中插入调用。_build_context() 的数据聚合与渲染逻辑分离，使同一份数据可用于不同渲染阶段。

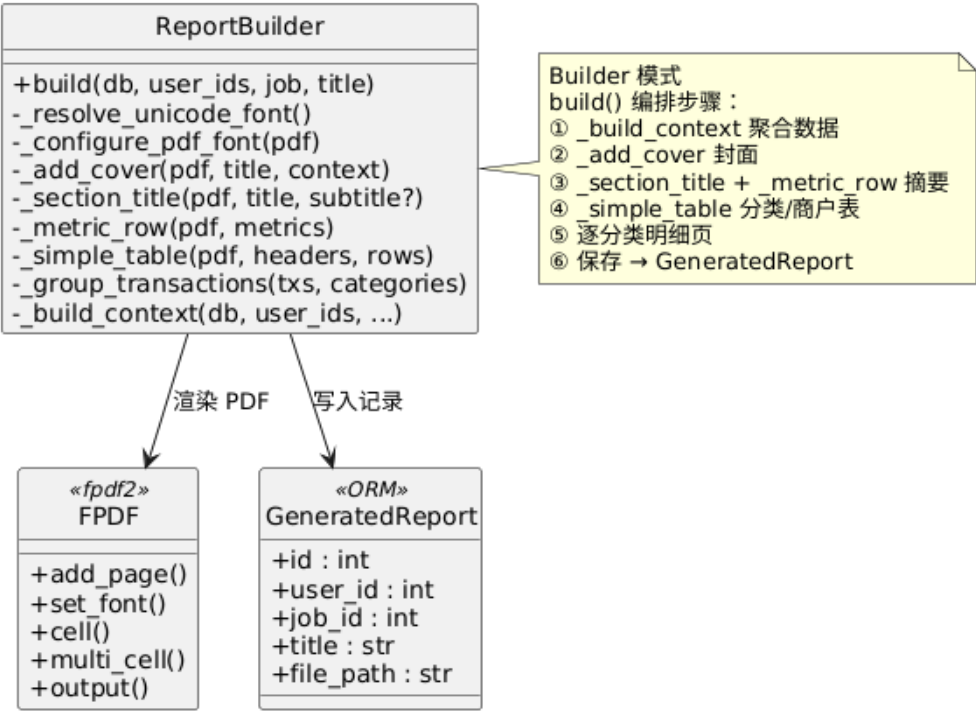


图 6.4 报表构建器类图

6.2 数据库设计

系统数据库共 12 张表，按业务域分两组展示。

6.2.1 核心账单与分类

核心域涵盖 users、categories、category_rules、import_batches、uploaded_files、transactions、classification_results、classification_caches 八张表。

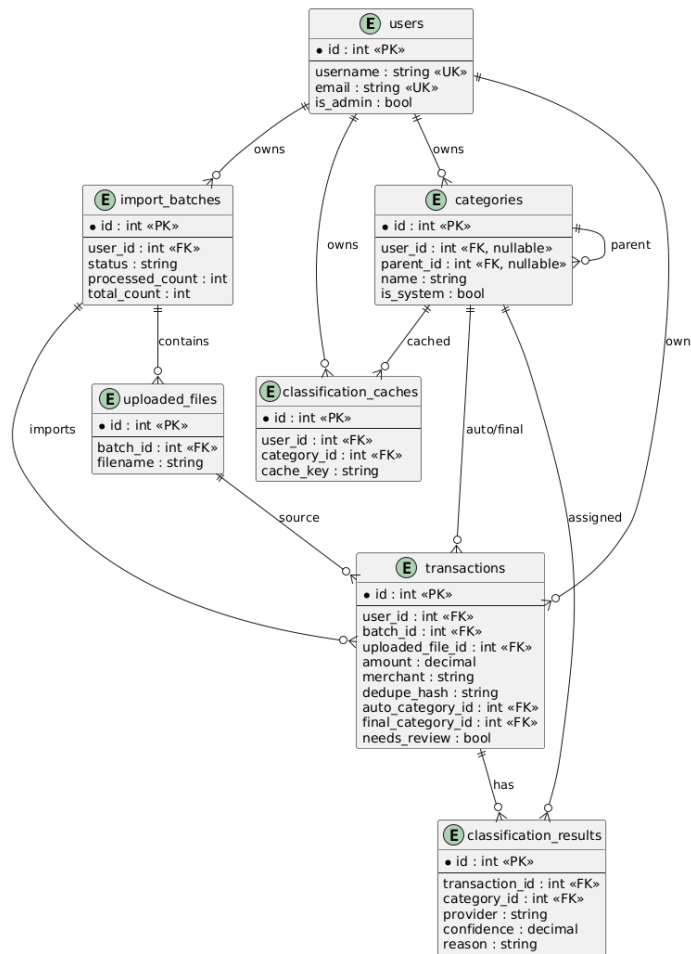


图 6.5 核心账单与分类 ER 图

关键设计决策: transactions 表以 (user_id, dedupe_hash) 联合唯一约束阻止重复导入; 双分类字段 (auto_category_id 与 final_category_id) 使模型建议与人工修正共存, 后者非 NULL 时优先; needs_review 布尔字段标识低置信度或未分类交易, 前端按此提取校正队列; classification_results 只增不改, 保留每次分类的完整记录用于追溯; classification_caches 以 (user_id, provider, text_hash) 唯一约束确保缓存按用户与 provider 隔离。

6.2.2 家庭组织与报表扩展

扩展域涵盖 organizations、organization_members、report_jobs、generated_reports 四张表 (users 与 transactions 与核心域共享)。

关键设计决策: organization_members.role 为枚举语义 (owner/admin/member), owner 拥有完整管理权限, admin 可管理普通成员, member 仅可查看聚合数据。家庭组织不改变交易归属——聚合查询通过收集 organization_members.user_id 列表并以 transactions.user_id IN (...) 过滤实现, 而非新增 organization_id 外键。report_jobs 记录生成请求的状态流转 (queued → processing → done/failed), generated_reports 记录输出文件路径。

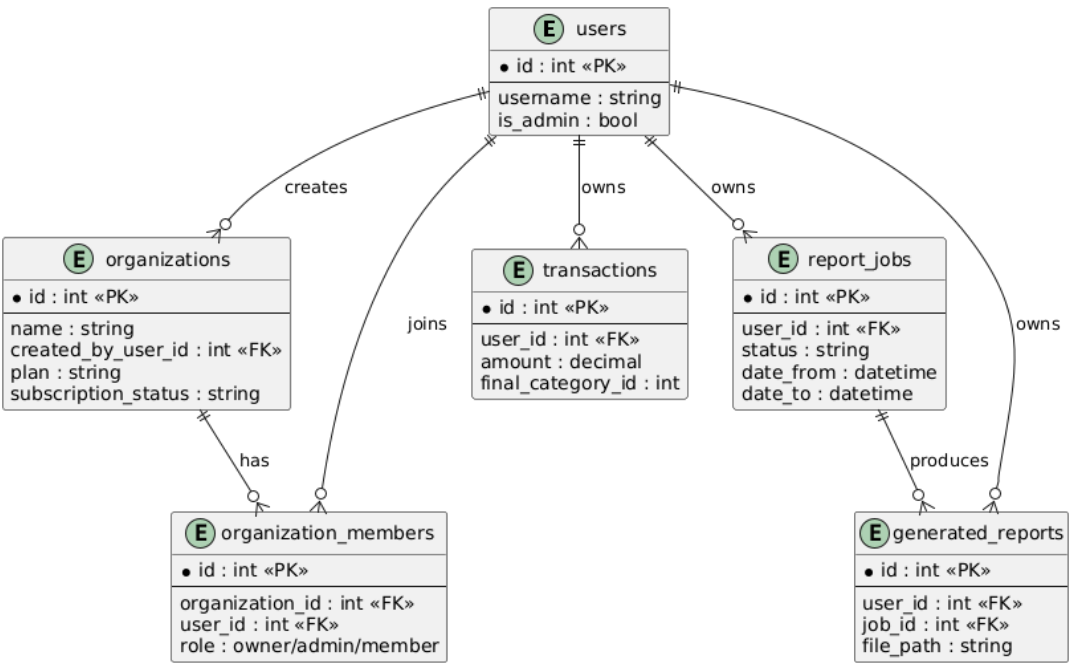


图 6.6 家庭组织与报表扩展 ER 图

6.3 设计模式应用

下表汇总系统在设计模式层面的应用情况，便于教师在单一位置审视 OO 设计的完整性。

表 6.1 设计模式应用汇总

设计模式	代码位置	说明	对应节
Data Mapper	models/entities.py (全部 ORM 类)	ORM 将领域实体与持久化解耦，实体不含 SQL	6.1.1
Mixin	entities.py:12 TimestampMixin	多重继承为 8 个实体横向复用时间戳字段	6.1.1
Strategy	classifiers.py:58,104,257 parsers.py:144,182	为接口下可互换的算法族，各自封装平台/provider 逻辑	6.1.2/6.1.3
Chain of Responsibility	classifiers.py:428 CompositeClassifier.classify	分类请求沿缓存 → API → 规则链路传递，不可用时自动降级	6.1.2
Facade	classifiers.py:267 CompositeClassifier	封装缓存、模型调用、重试入队、规则降级等复杂逻辑	6.1.2
Adapter	parsers.py:41 StatementReader	将 pandas 多编码/多格式 API 适配为统一 read 接口	6.1.3
Registry/Factory	parsers.py:239 ParserRegistry	根据文件名/内容识别平台，选择解析器并调度执行	6.1.3

接下页

表 6.1 设计模式应用汇总（续）

设计模式	代码位置	说明	对应节
Builder	reports.py:16 Report-Builder	将 PDF 多页渲染分解为独立方法，build() 按序列编排	6.1.4
Template Method	classifiers.py:115 OpenAICompatibleClassifier	classify() 定义 build→call→parse→return 固定骨架	6.1.2
DTO	schemas/*.py 全部 Pydantic 模型	请求验证与响应序列化，区分于 ORM 实体	5.7
Dependency Injection	dependencies.py FastAPI Depends	get_current_user、get_db_session 解耦认证与会话管理	5.7
Producer-Consumer	retry_queue.py 后台线程	串行消费 retry_queue 中的交易，调用外部模型	8.3
RBAC	organizations.py verify_org_* 函数	按 owner/admin/member 角色校验组织操作权限	5.5

6.4 核心流程设计

6.4.1 账单导入与大模型初分

用户上传账单文件的完整处理链路包含六个阶段。(1) 前端通过 POST /api/imports 上传文件，后端创建 ImportBatch 与 UploadedFile，前端轮询 GET /api/imports 跟踪批次进度。(2) ParserRegistry 检测平台后，由 StatementReader 读取文件，对应平台解析器将原生列映射为 ParsedTransaction。(3) TransactionNormalizer 做 NFKC 归一化并生成 dedupe_hash，写入前按唯一约束去重。(4) 每笔交易传入 CompositeClassifier.classify()，按缓存 → API → 规则的顺序获取分类结果。外部模型请求进入重试队列由后台线程串行消费，超时则指数退避重试。(5) 分类结果同时写入 transactions 表（更新 auto_category_id/confidence/provider/needs_review）和 classification_results 表（新增不可变历史记录）。(6) 全部交易处理完毕后，ImportBatch.status 更新为 done 或 partial_failed。

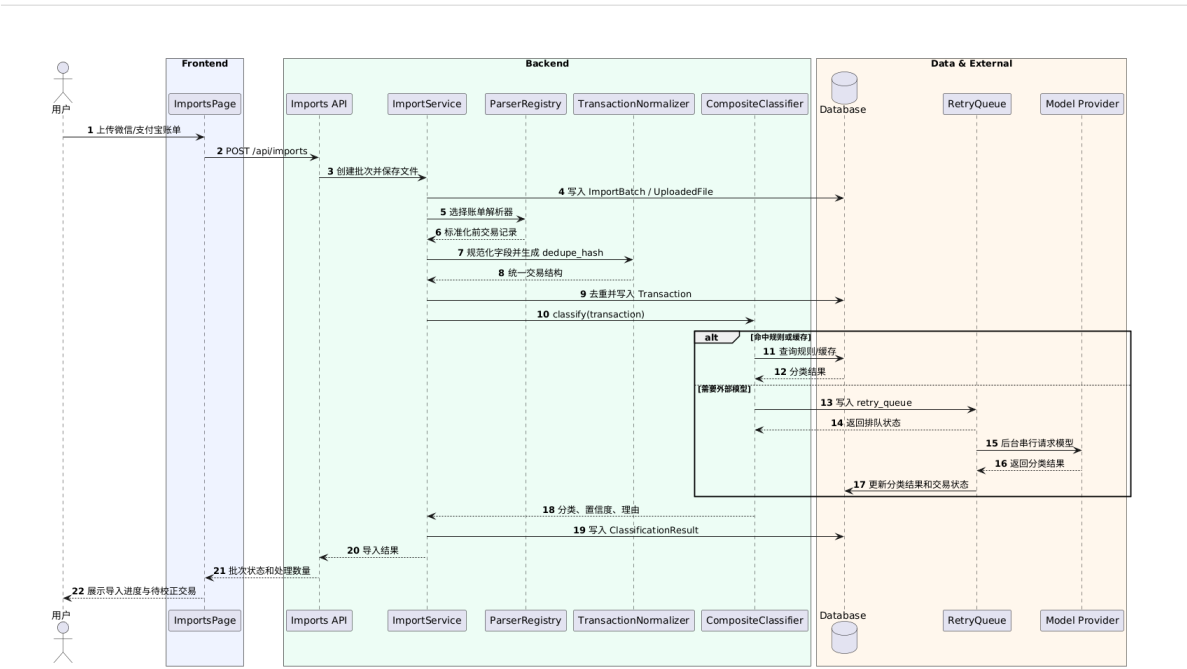


图 6.7 账单导入与大模型初分顺序图

6.4.2 家庭组织聚合查询

家庭组织视图的数据聚合不改变交易归属，在查询层面通过成员身份进行范围扩展。用户进入组织页面后，前端调用 `GET /api/organizations` 获取当前用户加入的所有组织及角色。查看家庭 Dashboard 或消费人格时，前端传入 `organization_id` 参数。后端先校验成员身份（非成员返回 403），再通过 `get_member_user_ids()` 收集全部成员 `user_id` 列表，传入对应的聚合服务。Dashboard 返回家庭总支出/收入/净流入与分类分布，Personality 基于全部成员交易计算家庭整体消费人格画像与财务健康评分，Reports 生成聚合 PDF 报表。

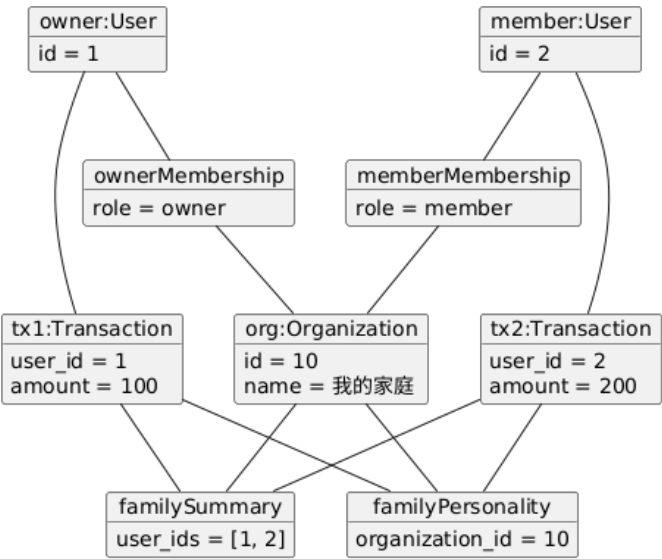


图 6.8 家庭聚合场景对象图

第七章 验收与发布

7.1 自动化测试

系统后端具备完整的自动化测试套件，位于 `backend/tests` 目录下，共 15 个文件（含测试辅助模块），其中 14 个为正式测试文件，覆盖安全、认证、导入、交易、分类、解析、分析、人格、组织、报表、可靠性与重试队列等核心领域。测试框架使用 `pytest`，可通过 `pytest` 或 `pytest --cov=backend` 运行。

表 7.1 自动化测试覆盖范围

测试文件	覆盖领域	测试数
<code>test_security.py</code>	密码哈希、Token 编解码	2
<code>test_api_auth.py</code>	注册、登录、登出 API	3
<code>test_api_imports_dashboard_reports.py</code>	导入、Dashboard 聚合、报表 API	4
<code>test_api_transactions.py</code>	交易查询与分类更新 API	4
<code>test_classifiers.py</code>	规则匹配、缓存、复合分类器	7
<code>test_parsers.py</code>	微信/支付宝账单格式解析	4
<code>test_analytics.py</code>	Dashboard 聚合计算	1
<code>test_personality.py</code>	人格维度计算、问卷对比、边界条件	2
<code>test_organizations.py</code>	组织创建、成员权限、家庭聚合、个人模式兼容	13
<code>test_reports.py</code>	PDF 报表生成	1
<code>test_reliability_metrics.py</code>	分类可靠性指标统计	3
<code>test_retry_queue.py</code>	重试队列入队、处理与状态查询	4
<code>test_normalizers.py</code>	文本规范化与哈希稳定性	2
<code>test_imports.py</code>	导入服务流程	3

7.2 测试结果

全部 53 个测试用例通过，后端整体覆盖率为 74%。各测试文件覆盖其对应服务或路由模块的核心逻辑路径。

前端构建测试包含两个阶段：TypeScript 类型检查（`vue-tsc --noEmit`）与 Vite 生产构建（`vite build`）。验证时通过 `npm ci` 安装依赖后执行 `npm run build`，两个阶段均通过，无类型错误或编译错误。Vite 构建输出提示部分 chunk 大小超过 500 KB，主要来源于 Element Plus 与 ECharts 组件库，属于后续性能优化项，不影响功能完整性。

表 7.2 测试结果汇总

测试类型	结果	说明
后端 pytest 单元/集成测试	53/53 通过	覆盖 14 个测试文件
后端代码覆盖率	74%	<code>pytest --cov=backend</code> 统计
前端 TypeScript 类型检查	通过	<code>vue-tsc --noEmit</code> 无错误
前端 Vite 生产构建	通过	静态资源构建成功
前端 Chunk 体积优化	待优化	Element Plus + ECharts 超过 500KB

7.3 验收测试

验收测试以手工联调方式执行，覆盖从注册到报表生成的完整用户流程，共 14 个验收步骤。

表 7.3 验收测试执行清单

步骤	操作	预期结果
1	注册新用户并登录	注册成功，跳转至 Dashboard
2	进入导入页面，上传微信或支付宝账单	文件上传成功，显示导入批次进度
3	检查最近导入结果与批次进度	批次状态从 processing 变为 done，交易计数正确

接下页

表 7.3 验收测试执行清单（续）

步骤	操作	预期结果
4	进入 Dashboard 查看概览	收入、支出、储蓄率、分类占比与 Top 商户正常显示
5	进入交易列表，按条件筛选	筛选结果正确，分页可用
6	打开交易详情抽屉	显示分类来源、置信度与分类理由
7	进入校正工作台，确认一笔待校正交易	交易从待校正队列移除，最终分类已更新
8	新增一个用户自定义分类	分类列表中显示新分类，校正工作台可选
9	进入报表中心，生成 PDF	生成成功，文件可下载，内容与交易数据一致
10	进入消费人格页面	人格画像卡片、四维雷达图、财务健康评分正常渲染
11	完成人格问卷自评	显示主观认知与交易数据画像的偏差对比
12	创建家庭组织并添加成员	组织创建成功，成员列表中显示被添加用户
13	查看家庭组织 Dashboard	显示家庭聚合收入、支出、交易数与家庭人格雷达图
14	查看系统设置页面	管理员可见 retry queue 状态与操作按钮，普通用户仅可见配置展示

每个验收步骤均在桌面视口（1440x900）下完成验证，确保页面无中文乱码、布局正常、交互响应正确。窄屏视口（390x844）下验证导航栏与内容区无严重重叠，表格与图表可正常展示。

7.4 发布状态

系统已完成部署并进入可用状态。部署架构采用 Docker Compose + Nginx 方案，各组件配置如下：

- **前端静态资源**：Vue 3 应用通过 Vite 构建为静态文件，由 Nginx 直接托管提供服务。
- **Nginx 反向代理**：监听 80 端口，前端路由由 Nginx 处理，/api 路径反向代理至后端 FastAPI 服务（127.0.0.1:8000）。
- **FastAPI 后端**：运行在内部 8000 端口，提供 RESTful API，由 Uvicorn 启动。
- **PostgreSQL**：持久化存储用户、交易、分类、组织、报表等业务数据。
- **Redis**：缓存与异步任务消息队列。低配部署时可通过 `TASK_ALWAYS_EAGER=true` 减少 Redis 依赖。
- **重试队列 Worker**：通过 FastAPI lifespan 事件启动后台线程，串行处理标记为 `retry_queue` 状态的交易分类请求。

管理员账号通过后端 CLI 工具创建，不提供公开注册入口。生产环境配置包括环境变量注入（数据库连接、模型 API 密钥、Redis 地址等），通过 Docker Compose 的 `env_file` 或裸机部署的环境变量文件管理。

系统当前版本已交付的完整功能包括：用户认证与数据隔离、微信/支付宝账单导入与解析、大模型语义分类与人工校正、Dashboard 财务分析、消费人格画像与财务健康评分、家庭组织管理与聚合分析、PDF 报表生成与下载、多供应商分类切换与重试队列容错。系统具备生产级部署能力，后端测试覆盖率达到 74%，前端构建通过类型检查与生产构建验证。

第八章 运行与维护

本章阐述系统的运行环境配置、性能管理策略、可靠性保障机制以及日常运维检查要点。

8.1 运行环境

系统的运行时组件栈自上而下由以下部分构成：

浏览器端 用户通过现代浏览器（Chrome/Firefox/Edge/Safari 最新两个主要版本）访问 Vue 3 构建的单页应用。前端仅需静态文件托管，无服务端渲染（SSR）需求。

Nginx 生产环境反向代理与静态文件服务。静态资源（HTML/CSS/JS/字体/图片）由 Nginx 直接返回，API 请求通过 proxy_pass 转发至后端 Uvicorn 进程。建议配置 gzip 压缩（application/javascript、text/css）、合理的缓存头（静态资源 Cache-Control: max-age=31536000，含 hash 的文件名天然支持缓存失效）以及 HTTPS 终端（通过 Let's Encrypt 等证书）。

FastAPI / Uvicorn 后端应用进程，监听 127.0.0.1:8000（不直接暴露于公网）。Uvicorn 的 worker 数量根据 CPU 核心数设置（建议 `workers = CPU_CORES * 2 + 1`），生产环境可通过 systemd 或 Docker 管理进程生命周期。

PostgreSQL / SQLite 主数据库。生产环境推荐 PostgreSQL 14+，轻量部署或开发环境可使用 SQLite（文件级数据库，无需独立进程）。

Redis Celery 消息代理与结果后端。轻量部署模式下（`TASK_ALWAYS_EAGER = true`）可省略。

Retry Queue Worker 后台线程，随 FastAPI 应用进程生命周期管理。负责按 `updated_at` 顺序串行消费待重试的外部模型分类请求。

Local Model Service / vLLM 可选的本地大模型推理服务，通过 OpenAI-compatible API 接口提供服务，推荐部署于独立 GPU 服务器或同一主机的独立进程中。

OpenAI-compatible API 外部大模型服务，通过 HTTPS 调用。支持任何兼容 OpenAI Chat Completions 接口的服务（如 DeepSeek、通义千问、Moonshot 等）。

文件存储方面，用户上传的账单原始文件存放于 `uploads/` 目录（按用户 ID 创建子目录），生成的 PDF 报表存放于 `reports/` 目录。建议定期清理超过 90 天的上传文件以控制磁盘占用。

8.2 性能管理

下表汇总了各核心组件的已知性能瓶颈、当前缓解措施及未来优化方向：

表 8.1 性能管理汇总

组件	瓶颈描述	当前缓解措施	未来优化方向
账单导入解析	解析为 I/O 密集型操作（读取 CSV/XLSX），单文件可含数千行	按文件逐个处理，定时提交避免长事务；跳行试探控制扫描上限	大文件异步处理，Celery Worker 并行解析
模型语义分类	外部 API 延迟（200–2000ms/次）为主要瓶颈	统一重试池串行化外部调用；Classification-Cache 缓存避免重复请求	引入语义相似度缓存；支持批量分类接口减少网络往返
Dashboard 聚合	需扫描用户全部交易记录用于分类分布与趋势计算	使用 in_() 过滤、索引覆盖；限定日期范围减少扫描量	引入物化视图或预聚合表，定时刷新统计
消费人格计算	Shannon 指数与余弦相似度涉及全量交易扫描	当前规模下（单用户数千笔）耗时百毫秒级，可接受	引入增量计算，新交易更新统计量而非全量扫描
PDF 报表生成	fpdf2 单线程，报表渲染为 CPU 绑定操作	典型报表（20–50 页）生成秒级，可接受；异步任务避免阻塞 HTTP	超大型报表分页并发生成后合并
前端加载性能	Element Plus + ECharts 合计超过 500KB（未压缩）	生产构建启用 tree-shaking 与 gzip	路由级代码分割，按页面懒加载组件库图表

8.3 可靠性保障

系统通过以下五项机制保障数据处理的可靠性：

1. **去重哈希 (dedupe_hash)**。每笔交易入库前，由 TransactionNormalizer 对平台、时间、金额（保留两位小数）、商户（NFKC 归一化后）、商品（NFKC 归一化后）五个字段拼接后计算 SHA-256 哈希。transactions 表的 (user_id, dedupe_hash) 联合唯一约束在数据库层面阻止重复插入。同一文件多次导入时，重复交易自动跳过，避免数据膨胀。
2. **统一重试池**。外部大模型 API 调用不直接由业务请求线程执行。CompositeClassifier 将需外部分类的交易标记为 auto_provider = "retry_queue" 后立即返回，由后台 RetryQueueWorker 线程按 updated_at

升序串行取出一条交易调用外部 API。重试采用指数退避策略：每次超时后 `api_retry_count` 递增，等待间隔依次增长。达到 `RETRY_QUEUE_MAX_RETRIES` 上限后，交易标记为 `retry_failed` 并从活跃队列中移除，不混入人工校正列表（避免用户在 Review 页面看到非自身错误导致的异常交易）。管理员可通过 `POST /api/classification/retry-all` 将失败交易重新入池。

3. **导入批次状态机。** 批次状态按 `queued → processing → done/partial_failed/failed` 流转。`progress_percent` 属性根据 `processed_count / total_count` 动态计算。前端轮询 `GET /api/imports` 获取批次列表，实时展示导入进度条与状态标签。`partial_failed` 状态表示部分交易处理失败但整体流程可继续。
4. **报表任务状态机。** 与导入批次类似，`ReportJob` 状态按 `queued → processing → done/failed` 流转。异步生成模式（Celery Worker）确保大报表生成不阻塞 API 线程，用户可在生成完成后下载。
5. **接口级权限校验。** 所有业务接口在执行查询前均通过 `get_current_user` 依赖注入验证身份，并在查询中添加 `user_id` 或组织成员身份过滤。资源归属检查覆盖交易查询、分类修正、批次删除、报表下载等所有变更操作。跨用户或非组织成员的访问尝试统一返回 403 Forbidden。

8.4 运维检查

日常运维中建议按照以下检查清单定期审查系统状态：

表 8.2 运维检查清单

检查项	操作方式	预期结果与说明
日志监控	查看 Uvicorn 访问日志与错误日志	关注 5xx 错误频率、超时日志、异常堆栈; 建议使用 <code>journalctl -u boring-financial</code> 查看 <code>systemd</code> 服务日志
健康检查	<code>curl http://localhost:8000/health</code>	返回 <code>{"status": "ok"}</code> 表示服务正常; 可作为监控系统的探活端点
数据库备份	<code>pg_dump boring_financial > backup.sql</code> (PostgreSQL) 或复制 <code>*.sqlite</code> 文件	生产环境建议每日自动备份, 保留最近 7 天
密钥轮换	修改 <code>.env</code> 中 <code>JWT_SECRET_KEY</code> , 重启服务	轮换后现有 token 失效, 用户需重新登录, 建议低峰期执行
依赖安全审计	<code>npm audit</code> (前端)、 <code>uv lock --check</code> (Python)	关注高危漏洞报告, 及时更新受影响的依赖版本
上传文件控制	检查 <code>MAX_UPLOAD_SIZE_MB</code> 等配置	确保符合安全策略, 定期检查 <code>uploads/</code> 目录
权限回归测试	<code>pytest tests/ -k "auth or organization" -v</code>	修改认证或组织代码后执行, 确保隔离与 RBAC 逻辑未退化
重试队列监控	<code>GET /api/classification/retry-status/</code> (需 admin token)	关注 <code>failed</code> 数量; 若持续增长, 检查外部 API 可用性
磁盘空间检查	<code>du -sh uploads/ reports/</code>	不应超过预期; 建议定时清理超过 90 天的上传文件

此外, 建议在生产服务器上配置以下监控:

- **进程存活监控:** 通过 `systemd` 的 `Restart=always` 或 `Docker` 的 `restart: unless-stopped` 确保 Uvicorn 和 Nginx 进程异常退出后自动重启。
- **内存与 CPU 监控:** 使用 `htop` 或 `Prometheus + Node Exporter` 监控服务器资源消耗, 关注内存泄漏与 CPU 尖刺。
- **磁盘 I/O 监控:** SQLite 在写入密集型场景下可能出现锁竞争, 若切换至 PostgreSQL 则可显著缓解。
- **外部 API 可用性告警:** 若 `Retry Queue` 的失败数量持续超过阈值, 应触发告警通知运维人员检查外部模型服务的可用性与配额状态。

第九章 总结与改进计划

9.1 项目完成情况

Boring Financial 智能账单分类系统在四个月的开发周期内，完成了从需求分析、系统设计、编码实现到测试部署的完整流程交付，兑现了项目启动阶段设定的核心目标。

在功能层面，系统实现了完整的账单分析闭环：用户可通过 Web 界面上传微信或支付宝账单文件，后端 ParserRegistry 自动识别平台格式并将其转换为统一交易结构，CompositeClassifier 按规则匹配、缓存查询、外部大模型 API 的顺序进行语义分类并输出分类理由和置信度，低置信度交易自动进入人工校正工作台供用户复核修正。基于分类后的交易数据，Dashboard 提供收入、支出、储蓄率、分类占比分布和收支趋势等核心指标的可视化看板；消费人格模块基于行为经济学理论（Laibson 双曲贴现模型、Thaler 心理账户理论、Veblen 炫耀性消费理论、大五人格 Openness 维度）生成 16 型消费人格画像和五维财务健康评分，并通过自评问卷与数据画像的偏差对比提供行为洞察。家庭组织模块在不改变个人账本归属的前提下，支持创建家庭、管理成员角色（owner/admin/member），并基于成员 user_id 列表聚合生成家庭 Dashboard、家庭消费人格和家庭报表。PDF 报表模块支持按日期范围和导入文件生成包含图表和统计表格的文档供下载。

在工程质量层面，后端实现了 8 个测试文件覆盖安全认证、文本规范化、分类器行为、导入服务、数据分析、消费人格、家庭组织和报表生成等核心模块。前端通过 TypeScript 类型检查和 Vite 生产构建验证了代码质量。系统支持本地直接启动、Docker Compose 编排部署和裸机 Nginx + systemd 三种运行模式，部署文档覆盖了从依赖安装到服务启动的完整流程。

在技术架构层面，前端采用 Vue 3 + Element Plus + ECharts 组件化方案，后端采用 FastAPI + SQLAlchemy + Celery 分层架构，通过 JWT 鉴权和 user_id 隔离实现多用户数据安全。外部模型请求通过统一重试队列串行处理，避免了高并发场景下的 API 调用风暴；规则分类和分类缓存作为兜底机制，确保在模型不可用时系统仍可正常运行。

9.2 当前局限与后续改进

系统在以下方面仍存在改进空间：安全层面，上传校验、速率限制、HTTPS 强制和安全响应头部尚未完善，refresh token 未实现服务端黑名单；测试层面，前端缺少端到端测试覆盖核心用户流程；性能层面，Dashboard 聚合在大数据量下依赖实时 SQL 扫描，前端打包体积偏大；运维层面，缺少 Alembic 数据库迁移管理和自动备份恢复机制。

后续改进将按优先级分步推进：短期聚焦安全加固（上传校验、速率限制、HTTPS）与前端 E2E 测试；中期引入 Dashboard 缓存层、路由级代码拆分与 Alembic 迁移工具；长期完善监控告警、自动备份与性能优化。系统的当前版本已形成功能完整、架构清晰、可部署运行的个人账单分类与分析产品。